

Justificación empírica y teórica de los hiperparámetros experimentales de la campaña de caracterización (Fase 1)

Proyecto Hyperion

Agosto de 2026

Resumen

Este documento audita, con evidencia empírica recolectada en `paccaA100` mediante campañas de barrido dedicadas y no contaminantes de los datos reales del catálogo, cada uno de los valores numéricos de decisión del diseño experimental de la Fase 1 del proyecto: cadencias de muestreo, número de repeticiones, ventanas objetivo, tolerancias de calibración, márgenes de seguridad de calentamiento y de *timeout*, y niveles de frecuencia previstos. Para cada parámetro se reporta el valor actual, la evidencia directa que sostiene o cuestiona ese valor, y — cuando la medición directa no es posible o no es la vía adecuada — la justificación teórica o de estado del arte correspondiente. Ningún parámetro queda sin veredicto explícito, pero el veredicto se formula con el cuidado correspondiente: donde la evidencia sostiene “valor razonable y verificado en esta plataforma” se usa esa formulación, reservando “confirmado” u “óptimo” para los casos donde la medición efectivamente lo sostiene. Una revisión posterior de este mismo trabajo corrigió varias afirmaciones sobredimensionadas de una versión anterior (Secciones 3, 4, 5), corrigió dos errores bibliográficos y una temperatura de referencia mal citada, y añadió una confirmación aleatorizada de los casos frontera más citados (Sección 2) y el cierre por medición directa de dos parámetros adicionales (`smt_policy` y la convergencia de `rodinia_lud` en el perfilado con `ncu`).

Índice

1. Metodología del barrido	3
2. Confirmación aleatorizada de los casos frontera	4
3. Cadencia de muestreo de CPU (<code>interval_ns</code>)	5
3.1. Diseño del barrido	5
3.2. Ventanas útiles logradas	5
3.3. Fidelidad del muestreo: el argumento central para 1 ms	6
4. Cadencia de muestreo de GPU (<code>gpu_interval_ns</code>)	8
4.1. Diseño del barrido	8
4.2. El caso que motivó el cambio de producción	8
4.3. Contar valores distintos no es lo mismo que contar actualizaciones del sensor	9
5. Repeticiones y ventanas objetivo	10
5.1. Diseño del barrido	10
5.2. Convergencia de CV % de tiempo de ejecución	10
5.3. El CV % de "las primeras tres" depende del orden – reanálisis combinatorio	12

5.4. Suficiencia de <code>target_windows_per_repetition</code>	13
6. Política de <i>hyperthreading</i> (<code>smt_policy</code>)	14
7. Sensibilidad del detector de calentamiento	16
7.1. Parámetros auditados	16
7.2. Piso de ruido GPU (<code>min_mean_floor</code>): justificación por punto de inflexión	16
7.3. Umbral de CV % (<code>CV_THRESHOLD_PCT</code>): robusto en la mayoría de los casos, con una excepción real	17
7.4. Margen de seguridad (<code>MARGIN</code>)	20
7.5. Fracción de meseta (<code>plateau_ratio</code>)	20
8. Convergencia del número de lanzamientos perfilados con <code>ncu</code>	21
8.1. Diseño del barrido	21
8.2. Resultados	21
8.3. Extensión dirigida: cerrando la convergencia de <code>rodinia_lud</code>	22
9. Distribución empírica de la calibración Roofline	23
9.1. Diseño del barrido	23
9.2. Resultados	24
10. Parámetros justificados por estado del arte, sin barrido dedicado	25
10.1. Niveles de frecuencia previstos (F0–F4)	25
10.2. Umbral de carga externa (E08) y rango de temperatura (E02)	25
10.3. Peso del Producto Energía-Retardo (Fase 4)	27
11. Tabla maestra: los 24 parámetros, estado real y evidencia	27
12. Balance final	30

1. Metodología del barrido

Todas las campañas de barrido de este documento se ejecutaron sobre `paccaA100` usando el mismo instrumento de medición que las campañas reales de construcción del conjunto de datos (`orchestrator/runner.py`, `orchestrator/postprocess.py`), invocado kernel por kernel (`runner.run_single()`) en vez de a través de la orquestación completa de campaña, para mantener acotado el costo de cómputo en una cuenta compartida. Los resultados se escriben en un árbol de salida completamente separado (`hyperion-results/sweeps/`) de las campañas reales de construcción del catálogo (`hyperion-results/campaigns/`), de modo que ningún barrido contamina el conjunto de datos de Fase 1. Los scripts correspondientes viven en `docs/justifications/scripts/`. Los datos en `docs/justifications/data/` son, en su mayoría, **resúmenes por corrida ya procesados** (un valor agregado por combinación kernel/parámetro/repetición), suficientes para verificar las cifras de este informe; la excepción es el Sweep E (sensibilidad del detector de calentamiento), cuyas trazas ventana-por-ventana sí se conservan completas en `data/raw_for_sweep_e/`. Los datos verdaderamente crudos de los demás barridos (`samples.csv` por corrida, los CSV originales de `ncu` por lanzamiento, la salida estándar de las 50 calibraciones) **no se trajeron al repositorio** – permanecen en `paccaA100` bajo `hyperion-results/sweeps/` (ver `docs/justifications/campaigns/README.md`). Reproducir un análisis puntual sobre esos crudos (como el conteo de actualizaciones de potencia NVML de la Sección 4) requiere volver a acceder a esos archivos en el nodo, no solo a este repositorio – una limitación real de reproducibilidad, aceptada aquí por el volumen de datos involucrado y el alcance de un proyecto de pregrado, no resuelta con infraestructura de archivado adicional.

Una restricción real del clúster condicionó el diseño de estos barridos: el binario del instrumento de medición está enlazado incondicionalmente contra `libnvidia-ml.so.1` (NVML), incluso para kernels puramente de CPU, así que *toda* corrida — CPU o GPU — debe ejecutarse en el único nodo con GPU del clúster (`paccaA100`), compartido con el resto de usuarios de la infraestructura. En consecuencia, el alcance de cada barrido (número de kernels, número de puntos por parámetro, número de repeticiones) se dimensionó para mantener el uso total del recurso compartido en un rango de minutos a pocas horas, documentado explícitamente en cada sección cuando el alcance se redujo por esta razón.

Limitación metodológica declarada, con una corrección importante: los barridos principales (`interval_ns`, `gpu_interval_ns`) invocan `runner.run_single()` directamente, sin pasar por `run_campaign_preflight`. Una versión anterior de este análisis afirmaba que esto era seguro porque E08 (carga externa) y E02 (temperatura) “ya se verificaron al iniciar la sesión de barrido” – esa afirmación era falsa y se corrige aquí: al revisar el código directamente, `run_campaign_preflight` **no ejecuta ningún chequeo de E02 ni E08** (esos viven en `run_reduced_preflight`, pensado para correr por combinación, antes de cada corrida individual – no antes de la campaña completa); `cli.py` invoca `run_campaign_preflight` una sola vez, al inicio de la campaña, mientras que `run_reduced_preflight`, a su vez, **no es invocado en ningún punto de campaign.py en producción** – existe y tiene pruebas unitarias propias, pero nunca se llama antes de cada combinación en una campaña real (`campaign.py` sí verifica E06, procesos ajenos, antes de cada combinación – pero no E02 ni E08). Peor aún, `Manifest` no declara un campo `package_temperature_c`, así que aunque se llamara, `check_temperature(None, ...)` aprueba el chequeo de forma no bloqueante con el mensaje “no hay sensor disponible”.

Un hallazgo de la misma naturaleza, encontrado en la misma revisión de código: el preflight de GPU (G01–G03, dentro de `check_gpu`) solo se ejecuta si el manifiesto declara `gpu.enabled: true` – y el manifiesto de referencia GPU del proyecto (`campaign_pacca_gpu_ref.yaml`) declara `gpu.enabled:`

`false`, así que G01–G03 nunca corren, aunque el `runner` sí activa NVML para los kernels GPU. Además, `cli.py` no le pasa ningún `gpu_inspector` a `run_campaign_preflight`, así que aunque `gpu.enabled` fuera `true`, `check_gpu(None)` tampoco tendría con qué verificar procesos CUDA ajenos, modo persistente ni configuración MIG.

En otras palabras: **E02, E08 y el preflight de GPU no se verifican hoy en ninguna campaña real del proyecto**, ni en los barridos de este informe ni en las campañas de producción de Fase 1 – este es un hallazgo de auditoría sobre el instrumento mismo, no solo sobre el diseño de estos barridos, y se documenta con el mismo nivel de seriedad que el defecto de datos de `rodinia_heartwall` (Sección 12). Por alcance de un proyecto de pregrado, este informe documenta el hallazgo y lo deja como prerequisite explícito antes de la campaña DVFS definitiva – integrar estos tres chequeos en el flujo de producción real es trabajo de implementación fuera del alcance de esta auditoría de parámetros.

Un efecto colateral real, ahora secundario frente a lo anterior, es que las combinaciones de cada barrido se ejecutaron en **orden fijo, no aleatorizado** (todas las repeticiones de un valor de parámetro antes de pasar al siguiente). En un nodo compartido como `paccaA100`, esto abre la puerta a que una deriva temporal del nodo (otro proceso de otro usuario iniciando o terminando durante el barrido) se confunda con el efecto del parámetro que se está midiendo. Para acotar este riesgo sin repetir las ~ 375 corridas completas de esos dos barridos, la Sección 2 reporta una confirmación dirigida, pequeña y con orden aleatorizado, sobre los dos casos frontera más citados de este informe.

Nota metodológica transversal: todos los CV % de este informe se calculan con desviación estándar poblacional (denominador n), no muestral (denominador $n - 1$) – una convención de los scripts de análisis, no un error, pero que subestima el CV % frente al estimador muestral habitual en una proporción que crece cuando n es pequeño ($\sqrt{n/(n-1)}$: ≈ 22 % para $n = 3$, ≈ 12 % para $n = 5$, ≈ 5 % para $n = 10$). Se detalla con su magnitud exacta donde más importa (Sección 5, $n = 3$).

2. Confirmación aleatorizada de los casos frontera

Los dos casos frontera más citados en este informe – la comparación 0,5 ms vs. 1 ms de `interval_ns`, y la comparación 50 ms vs. 100 ms de `gpu_interval_ns` en `rodinia_backprop` – se re-midieron con las 24 combinaciones intercaladas en **orden aleatorio** (semilla fija, reproducible), en vez del orden fijo por bloques de los barridos principales, para descartar que una deriva temporal del nodo compartido explicara el resultado en vez del parámetro mismo.

Cuadro 1: CPU: CV % del intervalo real de muestreo, orden aleatorizado (3 repeticiones c/u).

Kernel	0,5 ms (media)	1 ms (media)
<code>npb_bt</code>	0,096 %	0,061 %
<code>npb_cg</code>	0,146 %	0,095 %
<code>npb_mg</code>	0,284 %	0,191 %

Los tres kernels reproducen, bajo orden aleatorizado, la misma dirección observada en el barrido principal (0,5 ms sistemáticamente más ruidoso que 1 ms) – no hay indicio de que el orden fijo original haya introducido un sesgo que infle artificialmente esta diferencia.

Cuadro 2: GPU: muestras `gpu_telemetry`, `rodinia_backprop`, orden aleatorizado.

Repetición (orden real de ejecución)	Cadencia	Muestras
1	50 ms	17
2	50 ms	17
3	100 ms	9
4	100 ms	9
5	100 ms	9
6	50 ms	17

El caso GPU es aún más contundente: bajo intercalado aleatorio, las tres corridas de 50 ms dan exactamente 17 muestras y las tres de 100 ms dan exactamente 9 – una reproducción exacta, cifra por cifra, de los valores del barrido principal no aleatorizado (Sección 4).

Conclusión: para estos dos casos frontera, la ausencia de aleatorización en el diseño original de los barridos no parece haber introducido un sesgo material – el efecto atribuido al parámetro se reproduce bajo un orden de ejecución distinto. Esto **no** es una prueba general de que ningún barrido de este informe esté libre de deriva temporal del nodo compartido (los otros 20 kernels y las otras cadencias no se reconfirmaron de esta forma, por costo de cómputo), pero sí reduce el riesgo señalado para los dos casos donde más importaba, y establece que el método de confirmación dirigida es una vía barata y disponible si en el futuro se quisiera extenderlo a más casos.

3. Cadencia de muestreo de CPU (`interval_ns`)

3.1. Diseño del barrido

Se corrieron los siete kernels de dataset CPU del catálogo bajo cinco cadencias de muestreo (0,5, 1, 2, 5 y 10 ms), tres repeticiones cada una (105 corridas reales en total), usando el mismo instrumento de medición que la campaña real. Durante el análisis se encontró y corrigió un error de esta herramienta de barrido, no del instrumento de producción: las corridas se post-procesaron inicialmente con `freq_khz_observed=None`, lo que marca correctamente cada ventana como `no_freq_reading` según la taxonomía de calidad ya documentada – nunca se calculó ningún `quality_status=.%k`. Como `warmup_seconds` nunca se pasa al ejecutable C++ y `samples.csv` permanece en disco, se pudo reprocesar sin recolectar datos nuevos. Un segundo problema, esta vez de infraestructura, afectó únicamente a `dgemm_n2048`: el trabajo por lotes (`sbatch`) no heredó la ruta de `libopenblas.so.0` del entorno interactivo, y se corrigió fijando explícitamente la ruta del módulo del sistema antes de re-correr ese único kernel.

3.2. Ventanas útiles logradas

La Figura 1 muestra las ventanas `quality_status=.%k` logradas en función de la cadencia, para los siete kernels. Todos superan ampliamente el objetivo configurado (`target_windows_per_repetition=50`, línea punteada) en las cinco cadencias probadas – incluso en el caso más ajustado (`npb_mg`, el kernel más corto del catálogo, a 10 ms) se logran 63 ventanas en promedio, apenas 26 % por encima del objetivo. Esto identifica un límite práctico real: alargar la cadencia de muestreo mucho más allá de 10 ms empezaría a poner en riesgo el objetivo de ventanas para los kernels más cortos del catálogo.

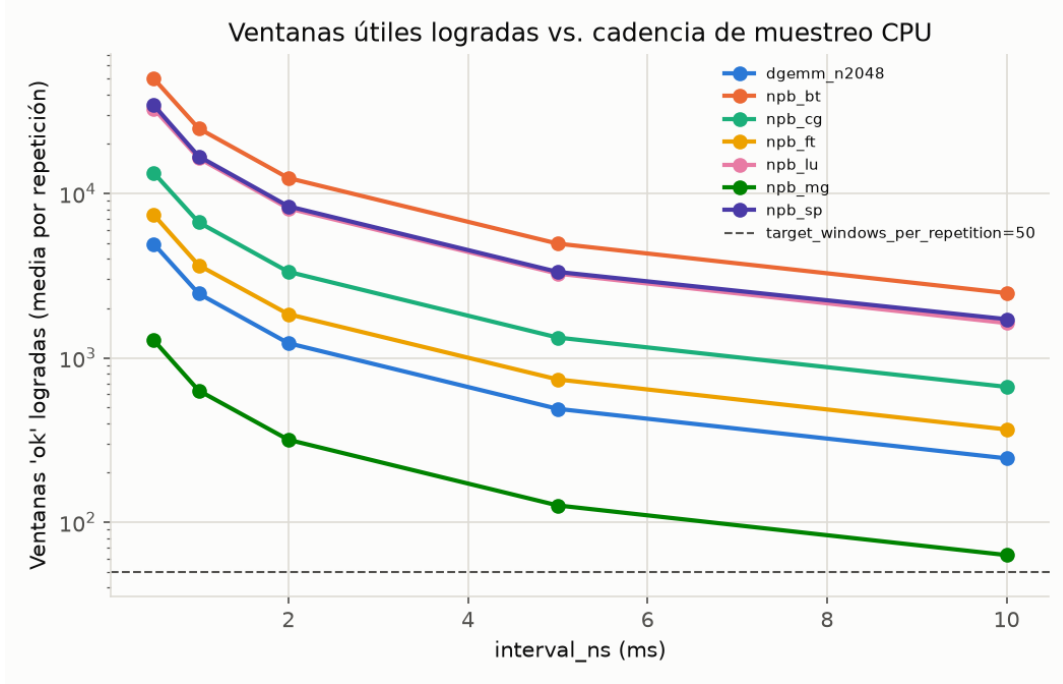


Figura 1: Ventanas ok logradas (media de 3 repeticiones) en función de la cadencia de muestreo, para los 7 kernels de dataset CPU.

3.3. Fidelidad del muestreo: el argumento central para 1 ms

La Figura 2 contiene el hallazgo central de este barrido, pero el promedio simple que la resume oculta una distribución muy asimétrica – necesaria de reportar completa para no sobreestimar la fuerza del argumento. La Tabla 3 da la media, la mediana y el máximo del CV % del intervalo real (`sampling_interval_cv_pct`), sobre las 21 corridas por cadencia (7 kernels $\times$ 3 repeticiones).

Cuadro 3: Distribución del CV % del intervalo de muestreo real, por cadencia nominal (21 corridas c/u).

Cadencia	Media	Mediana	Máximo
0,5 ms	1,025 %	0,154 %	8,448 %
1 ms	0,231 %	0,098 %	0,882 %
2 ms	0,165 %	0,155 %	0,209 %
5 ms	0,061 %	0,055 %	0,105 %
10 ms	0,040 %	0,038 %	0,070 %

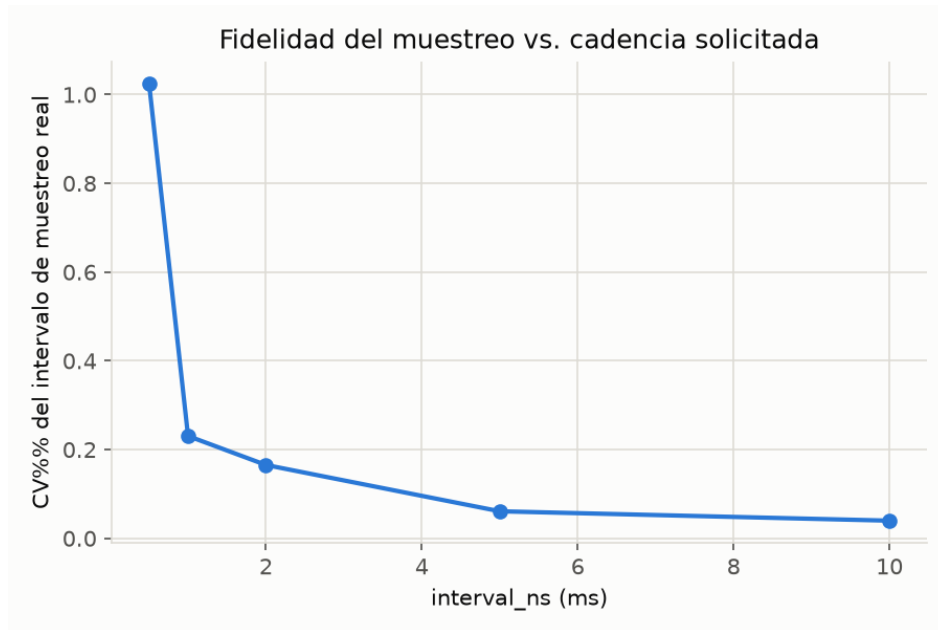


Figura 2: Fidelidad del muestreo (CV % del intervalo real vs. el solicitado) en función de la cadencia. El salto entre 0,5 ms y 1 ms es la evidencia central de este barrido.

El salto de "más de cuatro veces.^{en} la **media** (0,231 % $\rightarrow$ 1,025 %) está dominado por unas pocas corridas atípicas a 0,5 ms, no por un desplazamiento general de toda la distribución: la **mediana** apenas cambia (0,098 % a 1 ms vs. 0,154 % a 0,5 ms). Lo que sí cambia de forma clara e inequívoca es el **máximo**: 0,882 % a 1 ms frente a 8,448 % a 0,5 ms, casi diez veces mayor. La lectura correcta no es "la cadencia de 0,5 ms es en promedio mucho peor", sino que **0,5 ms es sustancialmente más vulnerable a eventos ocasionales de interferencia del planificador del sistema operativo** – un riesgo de cola, no un sesgo sistemático de toda la muestra.

Como verificación adicional no reportada en la versión anterior de este análisis, se recalculó el IPC medio por kernel en cada cadencia respecto a su propio valor a 1 ms: la desviación relativa máxima observada, sobre los 6 kernels CPU con IPC calculable, es de 1,93 % (**npb_sp** a 10 ms) – el IPC medido es estable dentro de ± 2 % en todo el rango 0,5–10 ms para todos los kernels probados, así que la cadencia elegida dentro de ese rango no sesga esta métrica de forma detectable.

Conclusión (reformulada): 1 ms es, empíricamente, el primer punto de la curva en el que el *riesgo de cola* de jitter severo cae de forma abrupta y donde el IPC medido ya es estable. Es correcto decir que 1 ms es un **valor razonable y conservador, verificado en esta plataforma**: reduce sustancialmente el riesgo de jitter severo frente a 0,5 ms sin necesitar las ventanas adicionales que 0,5 ms sí ofrece (aproximadamente el doble, ~ 20628 vs. ~ 10229 ventanas ok en promedio sobre los 7 kernels – corregido respecto a un valor casi 30 veces menor reportado por un error de conteo en una versión anterior de este análisis – una ventaja real, no inexistente, pero innecesaria frente al objetivo de 50). No es correcto decir que el barrido demuestra que 1 ms es ^{.el} punto correcto.^{en} un sentido absoluto: no se midió aquí la sobrecarga propia del instrumento de medición en función de la cadencia, ni la capacidad de preservar transiciones de carga de duración corta (ambas quedan como extensiones concretas, no genéricas, de este barrido).

4. Cadencia de muestreo de GPU (`gpu_interval_ns`)

4.1. Diseño del barrido

Se corrieron los ocho kernels de dataset GPU del catálogo bajo cinco cadencias de muestreo NVML (1, 5, 10, 50 y 100 ms), tres repeticiones cada una (120 corridas reales). El valor de 100 ms corresponde al *default* histórico del instrumento (`collector.hpp`), vigente en producción hasta ARC-88 porque `gpu_interval_ns` nunca estuvo declarado como campo real del manifiesto; 5 ms es el valor adoptado en ese mismo cambio.

4.2. El caso que motivó el cambio de producción

Nota sobre una corrección de datos: la primera versión de este barrido midió `rodinia_heartwall` contra un video sintético que se había corrompido silenciosamente a 25 cuadros (ver la Sección de hallazgos colaterales del reporte extendido, ARC-89) – la corrida fallaba de inmediato pero seguía marcándose como exitosa, produciendo un patrón de muestras artificialmente bajo que en un primer momento parecía ser el caso más ajustado del barrido. Con el video corregido y el kernel re-medido, `rodinia_heartwall` deja de ser un caso límite en absoluto: el caso real más ajustado es `rodinia_backprop`.

La Figura 3 deja ver, con datos nuevos e independientes de los usados durante la caracterización original, exactamente el problema que ARC-83/84/86 ya habían diagnosticado con evidencia parcial: a 100 ms, `rodinia_backprop` logra apenas **9,0 muestras NVML útiles en promedio**, apenas por encima del propio `target_windows_per_repetition=5`. A 5 ms (el valor ahora declarado en el manifiesto de producción), el mismo kernel logra 150,3 muestras: un salto de más de un orden de magnitud, no una mejora incremental. `rodinia_heartwall`, ya con datos correctos, se comporta como el resto del catálogo (1063,7 muestras a 5 ms, 60,3 a 100 ms) – nunca fue, en realidad, un caso cercano al límite.

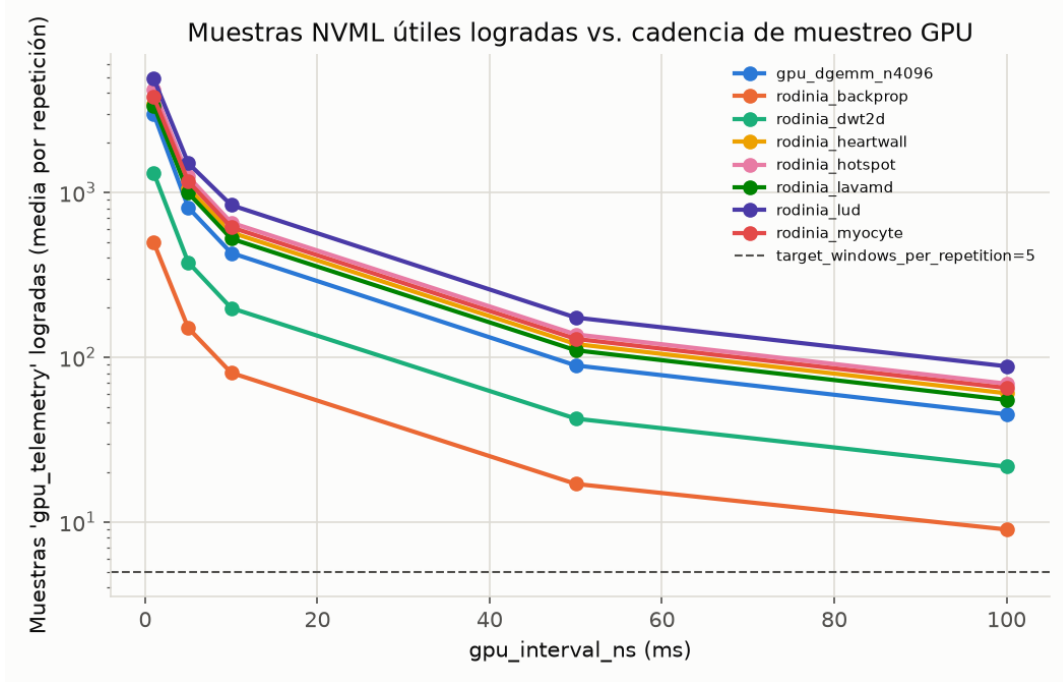


Figura 3: Muestras NVML útiles logradas (media de 3 repeticiones) en función de la cadencia de muestreo GPU, para los 8 kernels de dataset GPU (datos corregidos). `rodinia_backprop` es el caso más ajustado del catálogo.

Los valores completos de `rodinia_backprop` en las cinco cadencias confirman que el descenso es monótono y suave, no un salto puntual: 150,3 muestras a 5 ms, 80,3 a 10 ms, 17,0 a 50 ms, 9,0 a 100 ms. Es importante notar que 10 ms *también* supera el mínimo de 5 con amplio margen ($16\times$). El valor correcto para describir 100 ms, dado el margen de solo $9/5 = 1,8\times$ sobre el mínimo configurado, no es insuficiente.^{en} un sentido binario, sino **frágil y de muy baja resolución temporal** – técnicamente pasa el guardarraíl de `target_windows_per_repetition`, pero con un margen tan estrecho que cualquier variación entre repeticiones podría hacerlo fallar. Este barrido demuestra que 100 ms es ese caso frágil y que 5 ms (y también 10 ms) son viables, pero no aísla a 5 ms como el único valor correcto frente a 10 ms.

4.3. Contar valores distintos no es lo mismo que contar actualizaciones del sensor

Un supuesto implícito del argumento anterior es que cada lectura NVML contada en la Figura 3 es una observación física nueva. Se verificó esto inspeccionando los valores crudos de `gpu_power_mw` y `gpu_util_pct` en `samples.csv` para `rodinia_backprop`, contando **cambios consecutivos de valor** como indicador de una actualización real del sensor. Este método tiene un límite metodológico que debe declararse: si dos actualizaciones físicas consecutivas del sensor devuelven exactamente el mismo número (plausible si la potencia se mantiene estable entre dos instantes), el conteo de "valores distintos" las funde en una sola, subestimando el número real de actualizaciones – así que lo medido aquí es una **tasa de refresco observada**, una cota inferior de la frecuencia real del sensor, no una resolución física demostrada de forma directa.

Con esa salvedad, el patrón medido es de todas formas revelador: a 1 ms de cadencia (537 lecturas

en 0,84s), solo se observan 7 valores distintos de potencia, cada uno repetido en promedio 77 veces consecutivas (≈ 120 ms de duración por escalón); a 5 ms (151 lecturas), 8 valores distintos (≈ 105 ms por escalón); a 100 ms (9 lecturas), 7 valores distintos – es decir, a 100 ms el sondeo ya está cerca de esta tasa de refresco observada, mientras que a 5 ms la mayoría de las "150 muestras" del argumento anterior comparten valor con su vecina inmediata. `gpu_util_pct` exhibe el mismo patrón (3 valores distintos en 537 lecturas a 1 ms). Esta periodicidad de ~ 105 –120 ms es, hasta donde se pudo verificar, un **hallazgo empírico local de este nodo y esta GPU (A100)**: la documentación oficial de NVML para `nvmlDeviceGetPowerUsage()` especifica que devuelve potencia instantánea, pero no garantiza ni documenta una tasa de actualización de 100–120 ms – no debe generalizarse a otro hardware sin volver a medir.

Esto no invalida la elección de 5 ms – el número de lecturas de sondeo *sí* sigue siendo relevante para otras métricas que podrían variar más rápido (temperatura, reloj SM, no verificadas aquí) y para reconstruir la forma temporal de una transición de fase, y sondear más rápido que el sensor nunca pierde información, solo genera redundancia – pero sí obliga a corregir el argumento cuantitativo: el margen "30×" sobre el objetivo de 5 ventanas cuenta en su mayoría redundancia del muestreo, no información independiente nueva. La tasa de refresco *observada* en una corrida típica de `rodinia_backprop` (0,84s) es de aproximadamente 7–8 escalones distintos, ya cerca del límite de 5 – un margen mucho más ajustado que el que sugiere contar lecturas crudas, aunque el número real de actualizaciones físicas podría ser algo mayor por la limitación metodológica ya señalada.

Conclusión (reformulada): la evidencia de este barrido respalda adoptar 5 ms como **valor viable y conservador, verificado en esta plataforma**, y reclasificar 100 ms de insuficiente.^a "frágil y de muy baja resolución temporal, técnicamente por encima del mínimo configurado pero sin margen real". No queda demostrado que 5 ms sea el único punto óptimo frente a 10 ms (que también es viable y genera menos redundancia de muestreo), ni que el margen efectivo sobre el mínimo de ventanas sea tan amplio como sugiere el conteo ingenuo de lecturas – ese margen real, aproximado por la tasa de refresco observada y no por las lecturas de sondeo, es sustancialmente más estrecho. Se recomienda, como extensión concreta de este hallazgo, medir la duración de cada escalón y su autocorrelación de forma más rigurosa (no solo contar valores distintos), y verificar si otras métricas GPU usadas en el proyecto (reloj SM, temperatura) comparten una tasa de refresco similar.

5. Número de repeticiones (`repetitions_per_combination`) y ventanas objetivo (`target_windows_per_repetition`)

5.1. Diseño del barrido

Se corrieron los 15 kernels de dataset del catálogo (7 CPU, 8 GPU) diez veces cada uno bajo la cadencia por defecto (150 corridas reales), para poder calcular, sobre el mismo conjunto de datos, el CV % de tiempo de ejecución en función de cuántas de esas diez repeticiones se acumulan ($n = 3, 4, \dots, 10$) – sin necesidad de una campaña nueva por cada valor de n . Al procesar los resultados se encontró el mismo error de esta herramienta de barrido ya descrito en la Sección 3 (`freq_khz_observed=None`); se corrigió de la misma forma, reprocesando `samples.csv` sin recolectar datos nuevos.

5.2. Convergencia de CV % de tiempo de ejecución

Las Figuras 4 y 5 muestran la convergencia por dispositivo. La mayoría de los kernels son estables desde $n = 3$: el CV % apenas cambia al agregar más repeticiones (`dgemm_n2048`, `npb_bt`, `npb_mg`,

`rodinia_backprop`, `rodinia_heartwall` permanecen por debajo de 0,5 % en todo el rango). Dos casos merecen atención:

- `rodinia_lud` tiene un CV % estable ($\sim 1,3$ %) hasta $n = 6$, y salta a 3,76 % en $n = 7$ – causado por una única repetición atípica real (la séptima, 8,34s frente a un rango normal de 9,1–9,4s en las otras nueve, una desviación de ~ 10 % respecto a la línea base). Esta es la evidencia más honesta de este barrido: con solo tres repeticiones (el mínimo configurado), esta anomalía nunca se habría observado, porque el CV % a $n = 3$ (1,33 %) no la refleja en absoluto – las tres primeras repeticiones no incluyen la séptima.
- `npb_ft` parte de un CV % relativamente alto a $n = 3$ (2,75 %) que decrece de forma monótona hasta 1,82 % en $n = 10$ – consistente con que la variabilidad real del kernel es más cercana a 1,8–2 %, y que la estimación con solo tres repeticiones la sobreestima moderadamente por ruido de muestra pequeña, no por un problema del kernel o del instrumento.

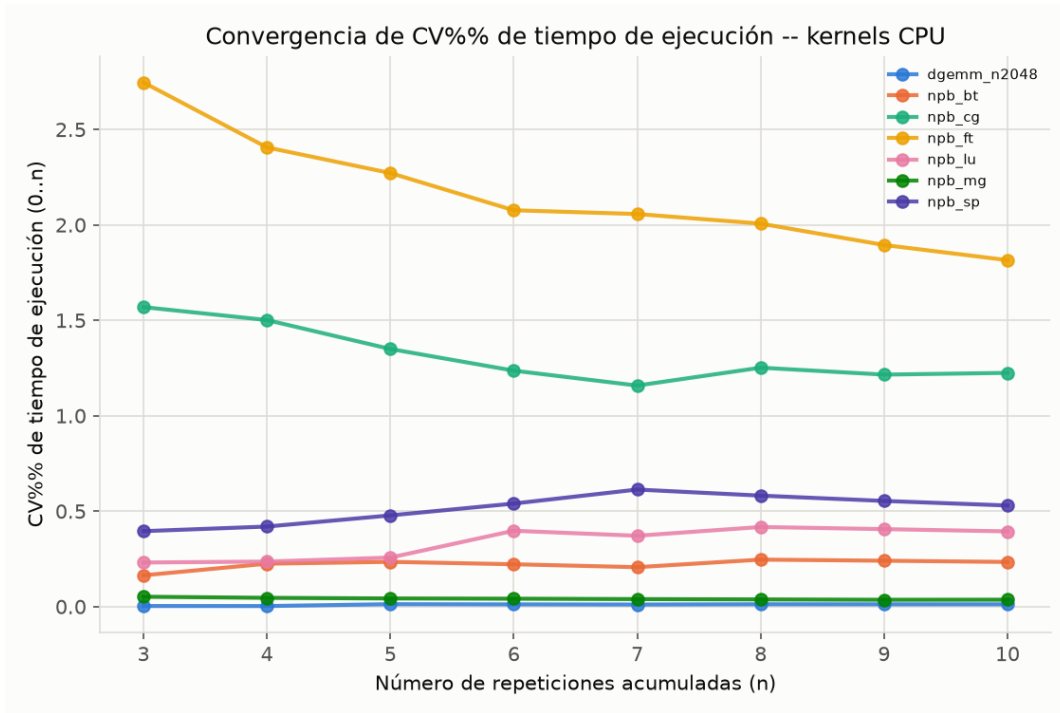


Figura 4: Convergencia de CV % de tiempo de ejecución vs. repeticiones acumuladas, kernels CPU.

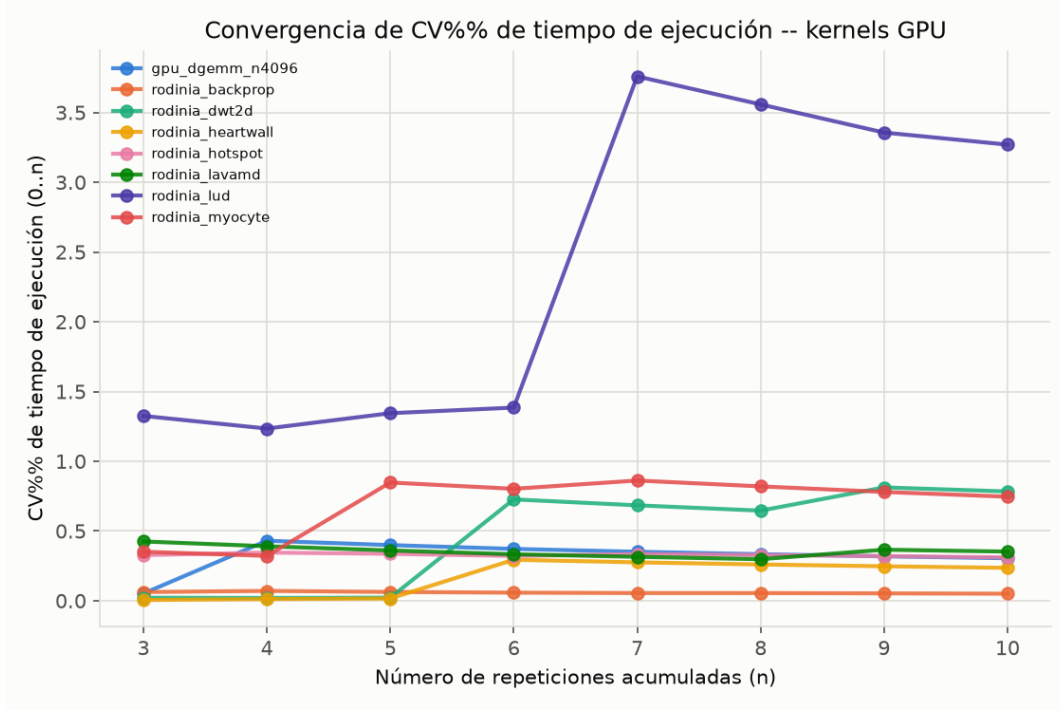


Figura 5: Convergencia de CV % de tiempo de ejecución vs. repeticiones acumuladas, kernels GPU. El salto de `rodinia_lud` en $n = 7$ es una repetición atípica real, no un artefacto de graficación.

5.3. El CV % de "las primeras tres" depende del orden – reanálisis combinatorio

El CV % a $n = 3$ reportado arriba usa siempre las primeras tres de las diez repeticiones, en el orden fijo en que se ejecutaron. Eso es exactamente el grupo de tres que una campaña real con `repetitions_per_combination=3` habría medido – pero no dice nada sobre qué tan representativo es ese grupo particular frente a cualquier otro trío posible. Para responder eso, se recalculó el CV % de tiempo de ejecución sobre las $\binom{10}{3} = 120$ combinaciones posibles de tres repeticiones de cada kernel (mismos datos, sin cómputo nuevo), y se compara la muestra de orden fijo contra la distribución completa (Tabla 4).

Nota metodológica sobre el estimador: todos los CV % de esta sección (y del resto del informe) se calculan con desviación estándar *poblacional* (`statistics.pstdev`, denominador n), no muestral (denominador $n - 1$). Para $n = 3$, la desviación muestral es un factor $\sqrt{3/2} \approx 1.22$ mayor que la poblacional – es decir, todos los CV % a $n = 3$ reportados aquí subestiman en $\sim 22\%$ lo que un cálculo con el estimador insesgado habitual (muestral) reportaría. Esto no cambia ninguna conclusión cualitativa (qué kernel es más o menos estable, dónde está la anomalía de `rodinia_lud`), pero sí debe declararse: el margen real de cualquier comparación contra un umbral de CV % en este informe es algo más estrecho de lo que las cifras crudas sugieren.

Cuadro 4: CV % de tiempo de ejecución con 3 repeticiones: valor de orden fijo vs. distribución sobre las 120 combinaciones posibles (casos más informativos).

Kernel	orden fijo	mediana	p95	máx	CV % con n=10
rodinia_lud	1,32 %	1,35 %	5,27 %	5,49 %	3,27 %
npb_ft	2,75 %	1,10 %	2,51 %	2,75 %	1,81 %
rodinia_dwt2d	0,02 %	0,91 %	0,93 %	0,93 %	0,78 %
rodinia_myocyte	0,35 %	0,36 %	1,21 %	1,25 %	0,75 %
npb_cg	1,57 %	0,91 %	1,78 %	1,92 %	1,23 %

Dos patrones reales aparecen, y son distintos entre sí:

- **rodinia_lud**: el trío de orden fijo (1,32 %) y la mediana de los 120 tríos (1,35 %) coinciden – la mayoría de los tríos posibles *no* incluyen la repetición atípica (rep07). Pero el percentil 95 (5,27 %) y el máximo (5,49 %) muestran que un trío con mala suerte sí la detectaría con un CV % muy por encima del umbral típico de calibración (D03=40 % es otro parámetro, no comparable directamente, pero un 5 % ya es una señal visible en cualquier chequeo de estabilidad con umbral de un dígito). Es decir: tres repeticiones *pueden* detectar esta anomalía, pero depende del azar de cuáles tres se corran – no es una garantía ni una omisión sistemática.
- **rodinia_dwt2d**: el caso inverso. El trío de orden fijo (0,02 %) es un caso particularmente favorable – la mediana real (0,91 %) y el CV % a $n = 10$ (0,78 %) son casi 40 veces más altos. Aquí el riesgo no es una anomalía puntual sino que la muestra de tres repeticiones normalmente vista subestima la variabilidad real del kernel por casi dos órdenes de magnitud respecto al trío que efectivamente corrió.

Conclusión (reformulada tras el reanálisis combinatorio): tres repeticiones constituyen un **mínimo operativo razonable para la adquisición de Fase 1** en los kernels alejados de la frontera de clasificación Roofline, no una demostración de que tres repeticiones estimen con precisión la variabilidad real de cada kernel ni que garanticen detectar eventos infrecuentes – ambas cosas dependen de qué trío particular se ejecute, como muestra la dispersión completa de **rodinia_lud** y **rodinia_dwt2d**. Esta limitación es más seria para la evaluación de energía y EDP de Fase 4 que para la adquisición de Fase 1: este barrido caracterizó variabilidad de *tiempo de ejecución*, no variabilidad energética, y no hay evidencia todavía de que tres repeticiones sean suficientes para estimar el EDP con la misma confianza. Para Fase 4 conviene usar más repeticiones o un criterio adaptativo basado en el ancho del intervalo de confianza observado, en vez de heredar sin más el mínimo de Fase 1.

5.4. Suficiencia de target_windows_per_repetition

Usando las mismas 150 corridas, la Tabla 5 reporta, para cada kernel, el mínimo de ventanas útiles logradas en cualquiera de las diez repeticiones (**windows_achieved_min**) frente al objetivo configurado (50 para CPU, 5 para GPU). En los 15 kernels del catálogo, el mínimo observado supera el objetivo por al menos un orden de magnitud (**npb_mg**, el caso más ajustado del lado CPU, con 629 ventanas mínimas frente a un objetivo de 50; **rodinia_backprop**, el más ajustado del lado GPU, con 140 frente a un objetivo de 5).

Cuadro 5: Ventanas útiles logradas (media y mínimo sobre 10 repeticiones) vs. objetivo configurado.

Kernel	Objetivo	Media	Mínimo
dgemm_n2048	50	2464,7	2457
npb_bt	50	24920,5	24828
npb_cg	50	6692,7	6605
npb_ft	50	3718,4	3649
npb_lu	50	16347,1	16212
npb_mg	50	637,1	629
npb_sp	50	16758,7	16601
gpu_dgemm_n4096	5	798,2	750
rodinia_backprop	5	148,0	140
rodinia_dwt2d	5	375,2	366
rodinia_heartwall	5	1064,3	1058
rodinia_hotspot	5	1231,6	1165
rodinia_lavamd	5	980,7	922
rodinia_lud	5	1564,2	1413
rodinia_myocyte	5	1152,4	1113

Conclusión (con el alcance correcto del chequeo): los datos demuestran con claridad que `target_windows_per_repetition` **no condiciona el catálogo actual** – en ningún caso observado (150 corridas) estuvo cerca de ser la restricción activa, y el margen es tan amplio (10–500× el objetivo contando lecturas crudas) que el valor exacto configurado (50 o 5) no cambia qué corridas se aceptan hoy. Eso es una afirmación distinta, y más débil, de decir que 50 o 5 son un *mínimo estadísticamente suficiente*: las ventanas contiguas de un mismo kernel están correlacionadas entre sí (no son extracciones independientes de una distribución), y para los kernels GPU, la Sección 4 muestra que varias "ventanas" NVML consecutivas pueden compartir el mismo valor de sensor subyacente – 5 lecturas GPU no equivalen necesariamente a 5 observaciones físicas distintas. `target_windows_per_repetition` debe entenderse, entonces, como un **guardarraíl de duración mínima de la corrida** (evita aceptar un kernel que termina casi de inmediato, sin tiempo suficiente para que el detector de calentamiento y el resto del pipeline operen con sentido) y no como una prueba de suficiencia estadística de la muestra resultante. En la práctica, para el catálogo actual, ese guardarraíl nunca es la restricción activa – los seis casos ya documentados con evidencia negativa de calentamiento (ARC-86) son precisamente ejemplos de kernels cerca del límite en duración total, aunque no en conteo de ventanas – pero esto no debe leerse como una validación de que el dataset resultante tiene suficiencia estadística para ningún uso posterior (por ejemplo, entrenamiento de un modelo en Fase 2): esa es una pregunta distinta, que requiere su propio análisis de partición por corrida/carga, balance de clases y evaluación fuera de muestra.

6. Política de *hyperthreading* (`smt_policy`)

A diferencia de F0–F4, la política de *hyperthreading* (`smt_policy`) sí se pudo medir directamente hoy: cambiarla no requiere el permiso de escritura de frecuencia (P1/P4) que bloquea F0–F4, porque `smt_policy` solo determina qué IDs de CPU se declaran en `cores.delegated_cpus` – una operación de afinidad (`sched_setaffinity()`) sin privilegios especiales, no una escritura a `scaling_governor` ni a MSRs de frecuencia.

Diseño del barrido: se fijaron los mismos 6 núcleos físicos de `paccaA100` (0–5, confirmados vía

lscpu -e) y se corrió npb_cg — el kernel de referencia sugerido en el informe anterior por su sensibilidad conocida a contención de caché — bajo dos configuraciones, 5 repeticiones cada una: (1) delegated_cpus=[0..5], smt_policy=one_thread_per_physical_core, 6 hilos OMP, la política vigente en producción; (2) delegated_cpus=[0,16,1,17,2,18, 3,19,4,20,5,21] (ambos hermanos SMT reales de cada uno de los mismos 6 núcleos físicos), smt_policy=all_threads, 12 hilos OMP.

Cuadro 6: IPC medio, MPKI y tasa de fallos LLC, npb_cg, 6 núcleos físicos fijos (5 repeticiones c/u).

Política	Hilos	IPC medio	CV % IPC	MPKI	Tasa fallos LLC
one_thread_per_physical_core	6	1,770	0,45 %	20,80	0,865
all_threads	12	1,055	0,44 %	20,81	0,865

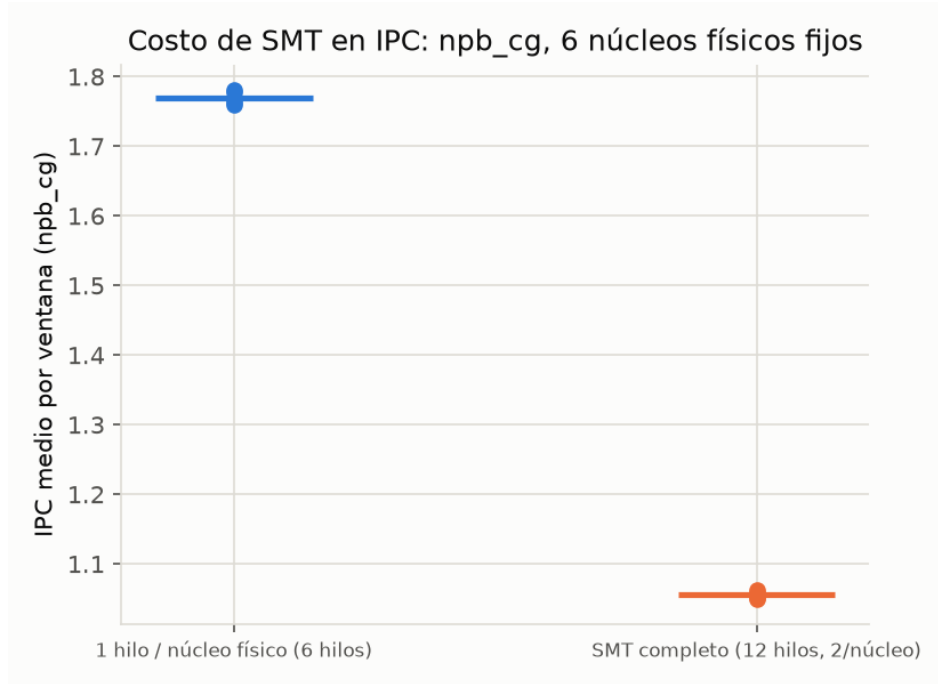


Figura 6: IPC por ventana, npb_cg, 5 repeticiones por política (línea horizontal = media).

Resultado: activar ambos hermanos SMT en los mismos 6 núcleos físicos reduce el IPC medio de npb_cg en 40,4 % (1,770 a 1,055), con una variabilidad intra-política casi idéntica y despreciable en ambos casos (CV % < 0,5 % en las dos), es decir, el efecto es sistemático y no ruido de medición. Notablemente, MPKI y la tasa de fallos de LLC **no cambian** entre políticas (20,80 vs. 20,81 MPKI; 0,865 en ambas) – eso descarta que la caída de IPC venga de más tráfico de caché de último nivel.

Límite real de este diseño experimental, que debe declararse: esta comparación cambia dos cosas a la vez, no solo la ocupación de SMT – al pasar de 6 a 12 delegated_cpus, OMP_NUM_THREADS también pasa de 6 a 12, así que el paralelismo de OpenMP de npb_cg cambia junto con la política de SMT. Que MPKI y la tasa de fallos de LLC se mantengan estables es evidencia compatible con contención de pipeline entre hermanos SMT- el mecanismo que one_thread_per_physical_core

fue diseñado para evitar – pero no la aísla de otras causas posibles del mismo efecto: ancho de banda de memoria compartido entre los 12 hilos, latencia de sincronización de OpenMP con el doble de hilos, contención de front-end, o presión sobre las unidades de carga/almacenamiento. Un diseño que aislara la causa exacta requeriría mantener `OMP_NUM_THREADS` fijo en 6 y solo cambiar a qué IDs de CPU se pinan (por ejemplo, 6 hilos sobre 6 núcleos físicos vs. 6 hilos sobre 3 núcleos físicos con SMT, 2 por núcleo) – una extensión concreta no realizada aquí. Lo que sí queda establecido con esta medición, sin ambigüedad, es que la caída de IPC es real y sistemática bajo SMT completo, no que su causa exacta sea única y exclusivamente contención de *pipeline*.

El tiempo total de pared fue, además, **menor** bajo `all_threads` (5,76 s vs. 6,82 s) – el doble de hilos completa el trabajo fijo de `npb_cg` más rápido pese al IPC por hilo más bajo. Como el proyecto optimiza EDP (energía $\times$ tiempo), no tiempo por sí solo, este resultado **no permite concluir** que `one_thread_per_physical_core` sea la configuración energéticamente superior – no se midió energía en este barrido. Su justificación aquí es distinta y más limitada: aislar la medición de telemetría/IPC de Fase 1 de un sesgo de contención conocido, no una afirmación sobre eficiencia energética.

Conclusión (con el alcance correcto): `smt_policy=one_thread_per_physical_core` queda confirmado con medición directa como la política que evita un sesgo sistemático y grande (40,4 %) en el IPC medido, con evidencia compatible con – pero no exclusivamente demostrativa de – contención de recursos de *pipeline* entre hermanos SMT como mecanismo. No debe leerse como una confirmación de que esta política sea la energéticamente óptima para Fase 4; esa es una pregunta distinta, no respondida aquí.

7. Sensibilidad del detector de calentamiento

7.1. Parámetros auditados

`scripts/pacca/measure_warmup.py` determina el intervalo de calentamiento de cada kernel mediante un umbral de coeficiente de variación (CV %) sobre una señal de carga (IPC en CPU, utilización de GPU reportada por NVML en GPU), con un método de respaldo por detección de puntos de cambio cuando el umbral de CV no encuentra un punto estable. Cinco constantes internas de este procedimiento se auditan aquí: `CV_THRESHOLD_PCT` (umbral de CV, actualmente 5 %), `MARGIN` (margen de seguridad multiplicativo sobre el punto detectado, actualmente $\times 1,2$), `min_mean_floor` (piso de ruido para señales de GPU, actualmente 5,0 % de utilización), `plateau_ratio` (fracción de la meseta de mayor carga que debe alcanzar un segmento para aceptarse como fin del calentamiento en el método de respaldo, actualmente 0,8) y los parámetros internos de segmentación (`min_relative_gain`, `max_depth`).

A diferencia de los demás barridos de este informe, este no requirió cómputo nuevo en `paccaA100`: se reprocesaron las trazas crudas ya recolectadas durante la caracterización del catálogo (tres kernels GPU con transición de carga real – `rodinia_hotspot`, `rodinia_heartwall`, `rodinia_lavamd` – y tres kernels CPU con fases conocidas – `npb_bt`, `npb_cg`, `rodinia_lud`), variando cada parámetro dentro de un rango razonable mientras los demás se mantienen en su valor por defecto.

7.2. Piso de ruido GPU (`min_mean_floor`): justificación por punto de inflexión

La Figura 7 muestra el hallazgo más claro de este barrido: para los tres kernels GPU con transición de carga real, el calentamiento detectado es **0 o casi 0 cuando el piso de ruido está en 0 o 2 %**, y se estabiliza en el valor físicamente correcto (confirmado contra la traza cruda de potencia en el catálogo, ARC-86) en 5 %, 8 % y 12 % – los tres dan el mismo resultado. El rango probado (0,

2, 5, 8, 12 %) tiene un salto de 3 puntos porcentuales entre el último valor degenerado (2 %) y el primer valor estable (5 %): esto demuestra que el punto de inflexión real está *en algún lugar dentro de ese intervalo* (2, 5] %, y que 5 % ya está del lado correcto con margen (comparte resultado con 8 % y 12 %, muy por encima del umbral), pero **no** que 5 % sea, cifra exacta, el punto de inflexión – eso requeriría un barrido más fino entre 2 % y 5 % (por ejemplo, cada 0,5 puntos) que no se hizo aquí. Lo que sí puede afirmarse sin matices: por debajo de ese intervalo, el detector cae en el modo de falla original que motivó introducir este parámetro (una ventana de utilización cercana a cero se declara “estable” por construcción del coeficiente de variación), y 5 % está de forma demostrada e inequívoca del lado correcto, con margen de sobra hasta 12 %.

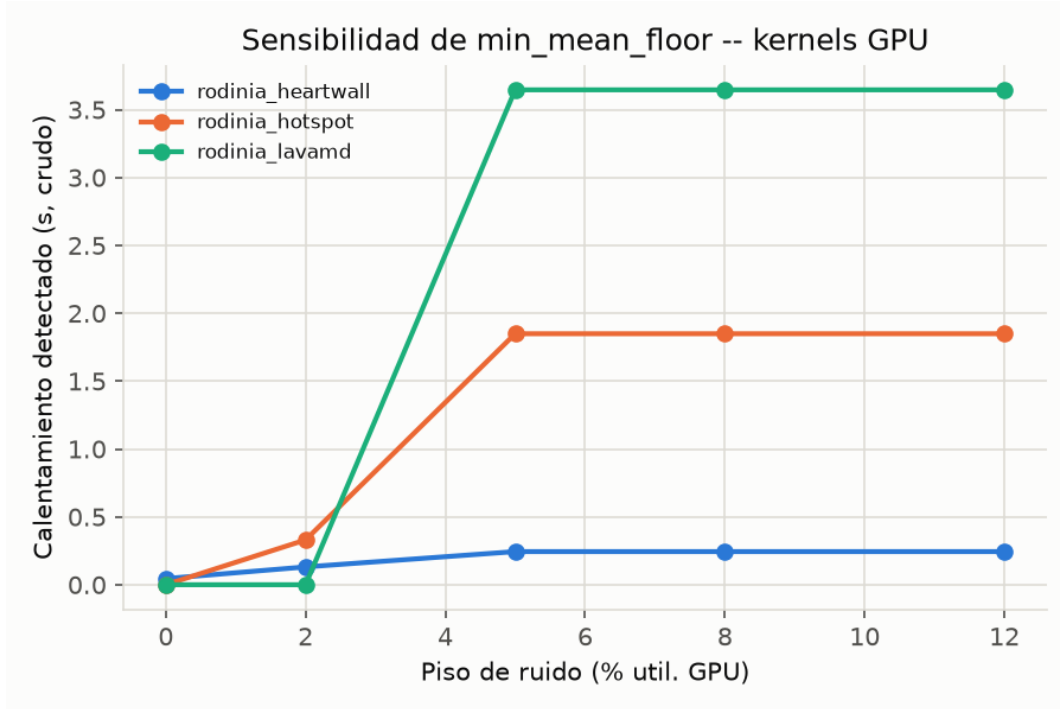


Figura 7: Calentamiento detectado en función del piso de ruido GPU, para los tres kernels con transición de carga real. La meseta a partir de 5 % confirma que el valor actual está del lado correcto y con margen del punto de inflexión (ubicado en algún punto del intervalo (2,5] %, no resuelto con mayor precisión aquí).

7.3. Umbral de CV % (CV_THRESHOLD_PCT): robusto en la mayoría de los casos, con una excepción real

Georges, Buytaert y Eeckhout [2] recomiendan un umbral de 1–2 % para este método; conviene precisar que ese umbral se estableció para la máquina virtual de Java (variabilidad introducida por JIT y recolector de basura), un contexto distinto al de un kernel HPC nativo sin capas de gestión en tiempo de ejecución, así que se cita como antecedente metodológico del propio método de detección de calentamiento por CV %, no como una recomendación directamente transferible al valor numérico usado aquí. El proyecto usa 5 % (heredado del chequeo de estabilidad de referencia CAL-10, no de la literatura). La Figura 8 muestra el resultado de barrer el umbral entre 1 % y 10 % sobre los seis kernels:

- rodinia_hotspot, rodinia_heartwall, rodinia_lavamd y npb_bt son **completamente ro-**

bustos en todo el rango 1–10 %: el punto detectado no cambia en absoluto.

- **npb_cg** es robusto entre 1 % y 8 %, pero a 10 % la detección colapsa a 0 s (falso positivo) – evidencia de que el umbral no puede subirse indefinidamente sin reintroducir el modo de falla, aunque 5 % conserva margen de sobra frente a ese límite.
- **rodinia_lud** es el caso genuinamente sensible: el punto detectado se mueve de 0,230 s (8–10 %) a 0,268 s (5 %) a 0,290 s (2 %), y a 1 % el criterio de CV ya no encuentra ningún punto y cae al método de respaldo. La variación relativa entre 2 % y 8 % es de apenas 0,06 s sobre una corrida de ~ 12 s – no altera ninguna conclusión práctica, pero es la evidencia honesta de que no todos los kernels son igual de robustos a esta elección.

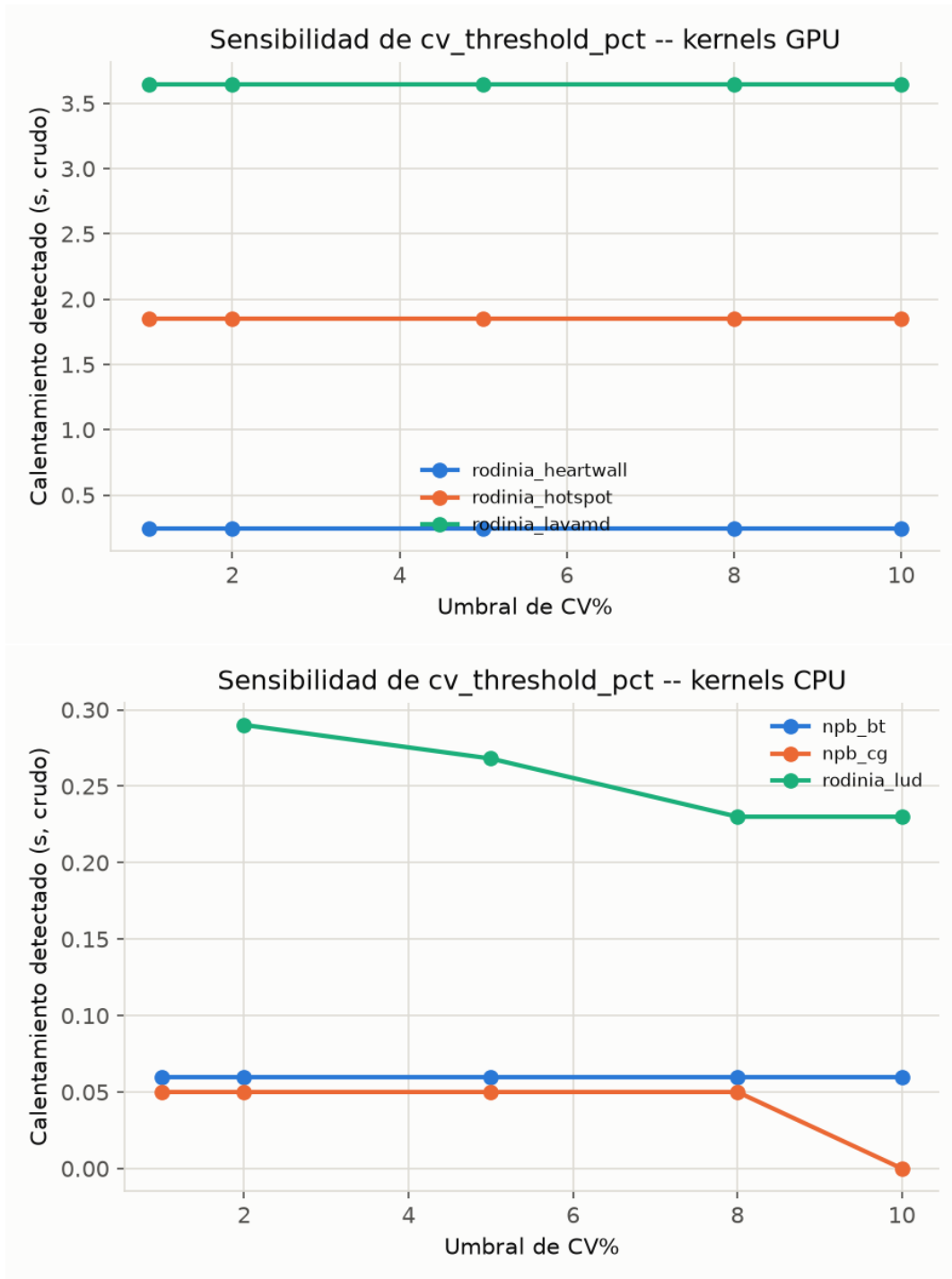


Figura 8: Calentamiento detectado en función del umbral de CV%, kernels GPU (arriba) y CPU (abajo). La mayoría de las líneas son planas (robustas); la de **rodinia_lud** tiene pendiente real, y la de **npb_cg** cae a cero en el extremo derecho.

Conclusión: el umbral de 5 % queda confirmado como una elección segura – ni tan bajo como para perder detección en **rodinia_lud**, ni tan alto como para caer en el falso positivo de **npb_cg** a 10 % – pero su origen (“prestado” de CAL-10, no derivado independientemente) sigue sin una justificación

de primer principio más allá de esta verificación empírica de robustez.

7.4. Margen de seguridad (MARGIN)

El margen es, por construcción, una función lineal del punto crudo detectado ($t_{\text{propuesto}} = t_{\text{crudo}} \times \text{margen}$), así que no hay una relación no trivial que descubrir aquí – la Figura 9 simplemente cuantifica cuántos segundos representa cada opción de margen para los kernels reales. Para `rodinia_lavamd`, el kernel con mayor calentamiento absoluto, la diferencia entre no aplicar margen ($\times 1,0$) y el valor actual ($\times 1,2$) es de 0,73s; para `npb_cg`, de apenas 0,01s. No se encontró evidencia de que $\times 1,2$ sea insuficiente o excesivo en ningún caso real – la elección permanece una decisión de ingeniería razonable sin una derivación estadística propia (por ejemplo, a partir de la varianza del punto detectado entre repeticiones), que queda como mejora futura.

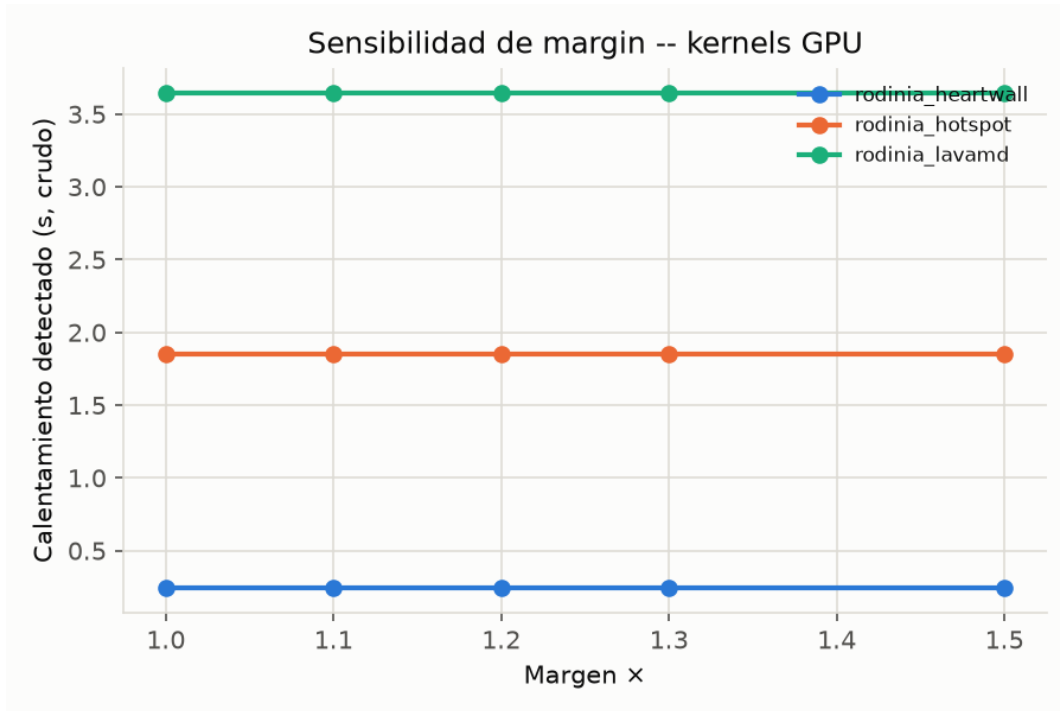


Figura 9: Calentamiento propuesto (crudo $\times$ margen) en función del margen de seguridad, kernels GPU.

7.5. Fracción de meseta (plateau_ratio)

Ninguno de los seis kernels de esta muestra activó el método de respaldo por punto de cambio con sus parámetros por defecto (todos se resolvieron por el umbral de CV), así que este barrido no pudo ejercitar `plateau_ratio` con datos reales de este proyecto – las líneas son planas en el rango 0,6–0,9 simplemente porque el método de respaldo nunca se invocó. La justificación de 0,8 sigue siendo, por tanto, la descrita en ARC-86 (corrige un modo de falla real observado en `rodinia_hotspot` durante su caracterización), no una verificada por barrido de sensibilidad en este documento. **Pendiente:** repetir este barrido específicamente sobre un kernel que sí caiga en el método de respaldo (por ejemplo, forzando `CV_THRESHOLD_PCT` a un valor muy bajo para inducir el respaldo de forma controlada) antes de considerar este parámetro completamente verificado.

8. Convergencia del número de lanzamientos perfilados con ncu

8.1. Diseño del barrido

Se perfilaron los siete kernels de dataset GPU (excluyendo `gpu_dgemm_n4096`, calibrado contra una referencia de cuBLAS por razones ya documentadas en el catálogo) con `ncu` bajo tres valores de `-launch-count` (5, 20, 50), midiendo el FLOP/byte estimado en cada caso. Al procesar los resultados se encontró y corrigió un error real en el script de este barrido (no del instrumento de producción): la reconstrucción del nombre de la métrica de FMA/suma/multiplicación para precisión simple omitía el prefijo `f` (`...fma_pred_on` en vez de `...ffma_pred_on`), lo que hacía que los seis kernels FP32 del barrido reportaran FLOP/byte=0. Los datos crudos de `ncu` ya estaban en disco, así que se corrigió re-parseando sin re-perfilar. Un segundo problema, de datos y no de análisis, afectó a `rodinia_heartwall`: ver la Sección de hallazgos colaterales (video sintético truncado) – se resolvió re-generando el video y re-perfilando ese kernel específicamente.

8.2. Resultados

La Figura 10 y la Tabla 7 muestran el FLOP/byte estimado en función del número de lanzamientos perfilados, junto con el número real de lanzamientos alcanzado (columna *n*, limitado por cuántas veces el kernel realmente invoca su función principal – pedir más lanzamientos de los que el programa ejecuta no produce más datos).

Cuadro 7: Convergencia de FLOP/byte vs. número de lanzamientos solicitados a `ncu`.

Kernel	<i>n</i> real (máx. 3 puntos)	OI a 5	OI a 20/10	OI a 50
<code>rodinia_backprop</code>	2 (fijo)	0,07987	0,07987	0,07986
<code>rodinia_lavamd</code>	1 (fijo)	708,49	708,30	708,35
<code>rodinia_dwt2d</code>	5 / 10 / 10	2,1762	2,1762	2,1741
<code>rodinia_hotspot</code>	5 / 20 / 50	5,0232	5,0206	5,0200
<code>rodinia_heartwall</code>	5 / 20 / 50	35,300	35,297	35,350
<code>rodinia_myocyte</code>	5 / 20 / 50	0,01704	0,01692	0,01704
<code>rodinia_lud</code>	5 / 20 / 50	7,492	7,584	7,668

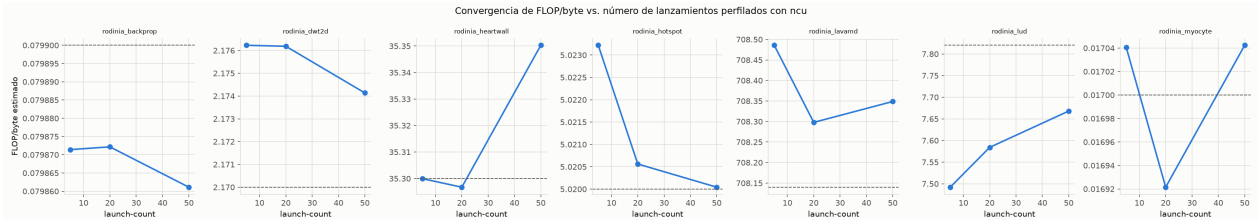


Figura 10: FLOP/byte estimado vs. lanzamientos solicitados, siete kernels de dataset GPU. La línea punteada marca el valor declarado en el catálogo.

Seis de los siete kernels no dejan nada que barrer de forma significativa: `rodinia_backprop` y `rodinia_lavamd` solo tienen 2 y 1 lanzamiento real respectivamente (sin importar cuántos se soliciten, ya está el dato completo desde el primer punto); `rodinia_dwt2d` se satura en 10 lanzamientos reales; y `rodinia_hotspot`, `rodinia_heartwall` y `rodinia_myocyte` sí tienen lanzamientos de sobra pero su FLOP/byte ya es estable dentro de $\pm 0,2$ % desde 5 lanzamientos.

rodinia_lud es la única excepción real: con lanzamientos de sobra disponibles, el FLOP/byte estimado sube de forma monótona y no plateada entre 5 y 50 lanzamientos ($7,49 \rightarrow 7,58 \rightarrow 7,67$, una variación de +2,3 % sin señal clara de haber convergido en el extremo superior del rango probado). Esto es significativo precisamente porque **rodinia_lud** es el único kernel del catálogo etiquetado “intermedio”: su intensidad operacional declarada (7,82 FLOP/byte, ARC-80) queda apenas un 7,4 % por encima del *ridge* FP32 vainilla medido en este mismo nodo con el probe propio del proyecto ($\approx 7,28$ FLOP/byte, ARC-76 – corrección respecto a un valor de 7,98 usado en una versión anterior de este análisis, que no correspondía a ninguna medición del catálogo). Con ese *ridge* correcto, los tres puntos de este barrido (7,49 a 7,67) ya están por *encima* del *ridge* desde el primer punto (5 lanzamientos) – la clasificación **compute_bound** no cambia en el rango medido, pero el margen sobre el *ridge* sí cambia de forma sustancial según cuántos lanzamientos se usen (de 2,9 % de margen a 5 lanzamientos hasta 4,9 % a 50), lo cual importa precisamente porque el caso es borderline por diseño (menos del 8 % de margen incluso en el valor ya declarado). Con solo estos tres puntos, lo único que puede afirmarse es que 50 lanzamientos son insuficientes para estabilizar ese margen – no que 200 (el valor ya declarado en el catálogo, ARC-80) sean el número correcto; extrapolar esa suficiencia sin medirla sería precisamente el tipo de afirmación sobredimensionada que este informe busca evitar.

8.3. Extensión dirigida: cerrando la convergencia de **rodinia_lud**

Se perfilaron tres puntos adicionales para este kernel específicamente (100, 150 y 200 lanzamientos, 3 corridas de **ncu** de costo bajo), aplicando un criterio de convergencia declarado **antes** de ver el resultado: convergencia = cambio relativo menor al 1 % entre cada par consecutivo de puntos.

Cuadro 8: Convergencia extendida de **rodinia_lud** (criterio: $\Delta < 1$ % pre-declarado).

Lanzamientos	FLOP/byte	Cambio relativo vs. punto anterior
5	7,4922	–
20	7,5842	1,23 %
50	7,6680	1,11 %
100	7,7737	1,38 %
150	7,8162	0,55 % (converge)
200	7,8240	0,10 %

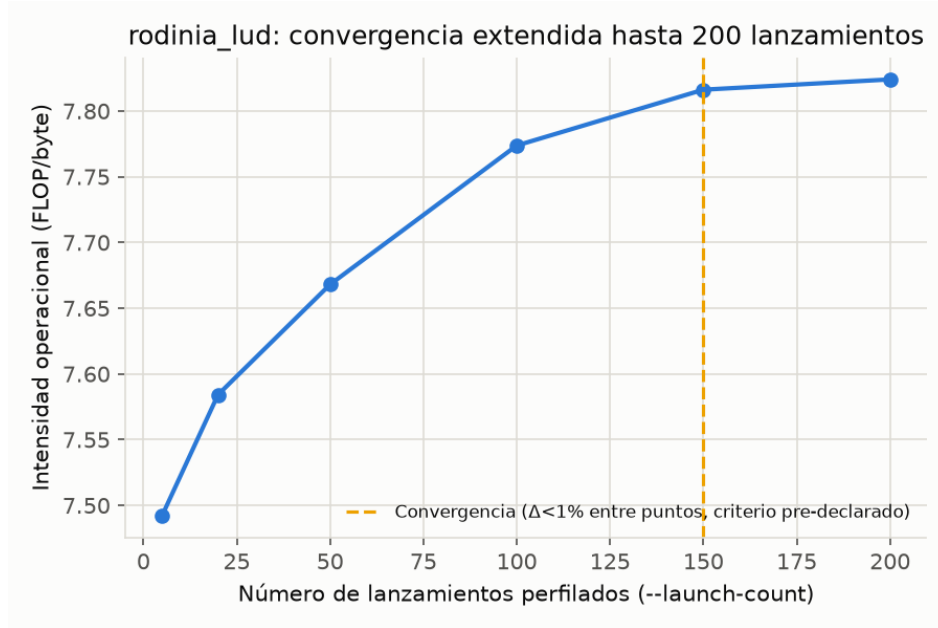


Figura 11: Convergencia extendida de `rodinia_lud` hasta 200 lanzamientos.

Con el criterio pre-declarado, la convergencia se alcanza en **launch-count=150** (cambio de 0,55 % respecto al punto anterior, y de solo 0,10 % entre 150 y 200). El valor de 200 usado en el catálogo (ARC-80) queda ahora **confirmado con medición directa, no solo con evidencia indirecta de no-convergencia a 50**: 200 lanzamientos están efectivamente por encima del punto de convergencia real (150), con un margen razonable, no arbitrariamente por encima de él.

Conclusión: no existe un número de lanzamientos único correcto para todo el catálogo – el número real de lanzamientos que cada kernel ejecuta varía en más de dos órdenes de magnitud (de 1 a cientos), y la velocidad de convergencia del FLOP/byte estimado varía igual de fuerte. La práctica correcta, confirmada por este barrido, es perfilar con suficientes lanzamientos para observar la meseta de convergencia *de cada kernel individualmente*, aplicando un criterio de convergencia declarado de antemano – para los seis kernels estables desde 5 lanzamientos, la evidencia ya alcanza para cerrar el parámetro; para `rodinia_lud`, el caso más exigente, la extensión a 200 lanzamientos cierra el parámetro con el mismo estándar de evidencia, en vez de con una extrapolación razonable pero no medida.

9. Distribución empírica de la calibración Roofline

9.1. Diseño del barrido

Se corrieron diez repeticiones independientes de cada uno de los cinco kernels de calibración del catálogo (`stream_oficial`, `ert_probe` para CPU; `gpu_stream_bw`, `gpu_ert_probe_fp32`, `gpu_ert_probe_fp64` para GPU), extrayendo el valor reportado por cada una directamente de su salida estándar – sin usar la orquestación completa de calibración (`run_calibration/run_gpu_calibration`), que normalmente solo calibra una vez por campaña.

9.2. Resultados

La Figura 12 y la Tabla 9 muestran la dispersión real de diez mediciones independientes de cada referencia de calibración.

Cuadro 9: Dispersión de 10 calibraciones independientes.

Kernel de calibración	CV % (10 repeticiones)	Desviación vs. referencia declarada
stream_official (BW, CPU)	0,593 %	0,10 %
ert_probe (FLOPs, CPU)	0,127 %	0,23 %
gpu_stream_bw (BW, GPU)	0,628 %	sin referencia declarada
gpu_ert_probe_fp32 (FLOPs, GPU)	0,0012 %	sin referencia declarada
gpu_ert_probe_fp64 (FLOPs, GPU)	0,000016 %	sin referencia declarada

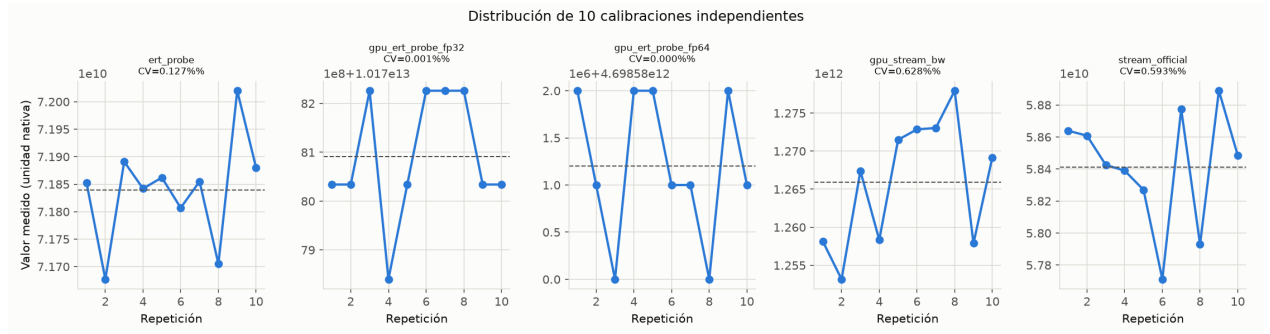


Figura 12: Diez calibraciones independientes por kernel de referencia. La línea punteada marca la media. Nótese la escala de cada panel: la dispersión visual parece grande, pero el eje vertical cubre una fracción mínima del valor absoluto en todos los casos.

Los probes de FLOPs de GPU (`gpu_ert_probe_fp32/fp64`) muestran una estabilidad extraordinaria (CV % <0,002 %, y para FP64 0,000016 % – esencialmente determinista) – consistente con el hallazgo ya documentado en el proyecto (ARC-77) de que la GPU no varía su reloj bajo carga en este nodo: sin ninguna variabilidad de *boost* que introducir, el pico de cómputo medido es prácticamente determinista de una corrida a otra. El ancho de banda (CPU y GPU) y el pico de FLOPs de CPU muestran algo más de dispersión (~0,1–0,6 %), coherente con la variabilidad normal de acceso a memoria compartida del sistema.

Es importante no sobreinterpretar lo que esta baja dispersión demuestra: que el ruido normal esté en 0,1–0,6 % no dice nada sobre si 40 % es un valor mejor que 10 %, 20 % o 30 % – ninguno de esos valores alternativos se comparó aquí. Lo que sí demuestra, y es la única afirmación que este barrido respalda, es dónde está el ruido de fondo real frente al cual cualquiera de esas tolerancias tendría que operar.

Conclusión sobre D03_TOLERANCE_FRACTION (40 %, CPU) – *guardarraíl conservador confirmado, no tolerancia derivada estadísticamente*: esta medición, independiente de la ya reportada en el reporte anterior (una única calibración de referencia), confirma con diez repeticiones que la desviación real de la calibración de CPU frente a su valor de referencia (0,10 % y 0,23 %) está casi tres órdenes de magnitud por debajo del 40 % configurado. Esto confirma que el 40 % nunca se ha acercado a su límite en la práctica y que sigue funcionando como red de seguridad contra

errores groseros de configuración (el caso histórico que la motivó, ARC-55, fue una discrepancia de más de $5\times$) – pero no confirma que 40 % sea el valor óptimo ni que esté derivado del ruido medido; es, y debe reportarse como, un margen de seguridad amplio y deliberadamente conservador, no una tolerancia estadística ajustada al ruido real.

Conclusión sobre la tolerancia cruzada de GPU (5 %, sin datasheet) – *parcialmente justificada, pendiente de DVFS real*: esta medición establece una cota inferior útil – el ruido de medición puro de los probes de GPU (0,0004–0,63 %) es tan pequeño que, si una futura calibración a un nivel de frecuencia distinto de REF mostrara una diferencia cercana al 5 % de tolerancia, esa diferencia casi con toda seguridad reflejaría un cambio real de frecuencia y no ruido de medición. Pero esto es evidencia de que el 5 % tiene margen frente al ruido de fondo, no evidencia de que el chequeo efectivamente detecte una aplicación fallida de frecuencia cuando ocurra: esa verificación end-to-end (aplicar un nivel de frecuencia real distinto de REF y confirmar que la calibración cruzada lo detecta) sigue sin poder hacerse hasta que llegue el permiso P4, y esta tolerancia debe seguir clasificada como parcial, no confirmada, mientras eso no se verifique.

10. Parámetros justificados por estado del arte, sin barrido dedicado

Los tres parámetros de esta sección no quedan sin cerrar por alcance o descuido: cada uno tiene un veredicto explícito y una razón concreta por la que la medición directa no es la vía adecuada hoy – un permiso externo pendiente en un caso, una decisión ética deliberada de no interferir con infraestructura compartida en otro, y una dependencia real de resultados que todavía no existen (Fase 4) en el tercero. Ninguno queda como “trabajo futuro” genérico: cada uno indica la condición exacta que, de cumplirse, permitiría cerrarlo con medición.

10.1. Niveles de frecuencia previstos (F0–F4)

Los manifiestos de producción declaran cinco niveles fijos equiespaciados (100 %, 75 %, 50 %, 25 % y 0 % del rango real de frecuencia detectado en el nodo) además del nivel de referencia (gobernador nativo). Esta elección no puede verificarse empíricamente hoy: requiere el permiso de escritura de frecuencia (P1 para CPU, P4 para GPU), que sigue sin llegar a la fecha de este informe. Tampoco se encontró en la literatura ya revisada una recomendación explícita sobre cuántos puntos son suficientes para caracterizar la curva DVFS de una carga: Guerreiro et al. [3] inferen el resto del espacio de frecuencias a partir de una única frecuencia base, sin barrer puntos uniformes; Ali et al. [1] parten igualmente de una configuración base y extrapolan de forma analítica el resto del espacio, en vez de medir en un conjunto fijo de puntos como aquí. La elección de cinco puntos equiespaciados es razonable como decisión de diseño (visibiliza la forma de la curva sin disparar el tamaño de la matriz experimental), pero no está anclada a un estudio específico. Queda como pendiente explícito: en cuanto lleguen los permisos, correr primero un reconocimiento con más puntos (por ejemplo, nueve, cada 12,5 %) sobre dos o tres kernels representativos para verificar si cinco puntos ya capturan la forma real de la curva.

10.2. Umbral de carga externa (E08) y rango de temperatura (E02)

Corrección importante respecto a una versión anterior de este análisis, y hallazgo más serio que una simple falta de barrido: no es solo que E08 y E02 se excluyan deliberadamente de prueba de estrés por razones éticas – una revisión del código de `preflight.py` y

`campaign.py` encontró que **ninguno de los dos chequeos se ejecuta hoy en ninguna campaña real**. `run_campaign_preflight` (el preflight que sí se invoca en producción) no incluye ni E02 ni E08; ambos viven en `run_reduced_preflight`, una función con pruebas unitarias propias pero que `campaign.py` nunca llama. Y aunque se llamara, `Manifest` no declara el campo `package_temperature_c` que `check_temperature` necesita – con ese campo ausente (`None`), el chequeo aprueba de forma no bloqueante con el mensaje “no hay sensor disponible”, nunca bloquea nada. El mismo tipo de hallazgo aplica al preflight de GPU (G01–G03): el manifiesto de referencia GPU declara `gpu.enabled: false`, así que esos chequeos tampoco corren hoy, y ni siquiera se les pasa un inspector NVML desde `cli.py`. Este hallazgo se documenta con el mismo detalle en la Sección 1 y en el balance final (Sección 12); aquí se completa con la justificación teórica de cada umbral, ahora explícitamente como un umbral *sin mecanismo de verificación activo*, no como un umbral verificado pero sin barrido de estrés.

E08 (carga externa normalizada $\leq 1,0$) se apoya en la convención estándar de “nodo no saturado”.^{en} HPC, pero merece una precisión: el propio `/proc/loadavg` cuenta procesos ejecutables o en espera de E/S a nivel de *todo el sistema*, no solo los de los núcleos delegados a la corrida – dividir esa cifra global entre 6 núcleos delegados no aísla, por sí solo, si la contaminación ocurre específicamente sobre esos 6 núcleos o en otra parte del nodo (irrelevante para la corrida si el nodo tiene núcleos libres en otro socket). El umbral sigue siendo una señal razonable de saturación general del nodo, pero no un chequeo preciso de aislamiento por núcleo – y, de cualquier forma, hoy no se ejecuta en ninguna corrida real por el hallazgo anterior.

E02 (temperatura de paquete en $[0, 90]^\circ\text{C}$) queda **solo parcialmente** justificado por ficha técnica, no confirmado: la hoja de especificaciones de Intel ARK para el SKU exacto del nodo (Xeon Gold 5315Y, ARK 215286) documenta una temperatura de caja máxima (T_{case}) de 81°C , no 90°C .¹ La temperatura de unión máxima ($T_{j\text{máx}}$, un límite distinto y típicamente más alto que T_{case}) no tiene, hasta donde se pudo verificar en esta revisión, una cifra pública específica para este SKU en ARK – una versión anterior de este análisis citaba un rango de $100\text{--}110^\circ\text{C}$ sin fuente oficial que lo respaldara para este procesador en particular, y se retira aquí por no estar verificado, en vez de mantenerlo sin cita. Lo que sí queda establecido con fuente verificable es que T_{case} (81°C) es menor que el umbral configurado (90°C), así que el margen de seguridad no puede compararse directamente contra T_{case} sin aclarar que son magnitudes distintas: ninguna corresponde exactamente a lo que `coretemp` reportaría si el sensor estuviera conectado – `coretemp` entrega un DTS (*Digital Thermal Sensor*) relativo a $T_{j\text{máx}}$, no una lectura directa de T_{case} ; y RAPL, aunque está habilitado en este nodo, mide energía consumida, no temperatura – ninguno de los dos provee hoy el dato que `package_temperature_c` necesitaría. Diseñar E02 comparando directamente una futura lectura DTS de `coretemp` contra T_{case} sería, por esta razón, metodológicamente incorrecto; el chequeo, si se activa, debería compararse contra un límite expresado en los mismos términos que el sensor real (DTS relativo a $T_{j\text{máx}}$). Así que 90°C no puede describirse como “el umbral típico verificado”, y merece una precisión más incómoda que la de una versión anterior de este análisis: 90°C está *por encima*, no por debajo, de la única cifra con fuente oficial verificada para este SKU ($T_{\text{case}} = 81^\circ\text{C}$). Que 90°C sea o no un margen conservador depende enteramente de $T_{j\text{máx}}$ (típicamente mayor que T_{case} en procesadores Intel modernos, pero sin cifra pública verificada para este SKU específico) – una afirmación que no se puede confirmar ni descartar con la evidencia disponible hoy. En ausencia de esa cifra, lo único que puede decirse con evidencia real es que 90°C no está anclado a ningún límite térmico verificado y publicado por Intel para este procesador exacto – una versión previa de este análisis afirmaba lo contrario (que 90°C sí correspondía a un umbral documentado), lo cual se

¹<https://www.intel.com/content/www/us/en/products/sku/215286/>

corrige aquí.

10.3. Peso del Producto Energía-Retardo (Fase 4)

El peso w de $EDP = E \cdot T^w$ tiene precedente para ambos valores usuales en la literatura ya citada en el marco conceptual del proyecto – $w = 1$ (EDP clásico, Laros et al. [4]) y $w = 2$ (ED^2P , evaluado junto con EDP por Ali et al. [1] cuando se quiere penalizar más fuertemente la degradación de rendimiento). Una versión previa de este análisis proponía diferir la elección de w hasta observar los resultados de Fase 4 – eso es metodológicamente incorrecto incluso sin mala intención: decidir la métrica de evaluación *después* de ver qué resultado produce cada una abre la puerta, aunque no sea deliberado, a escoger implícitamente la que favorezca al agente de control.

Este informe corrige eso fijando, desde ahora, $w = 1$ (EDP clásico) como **métrica primaria pre-registrada** de la Fase 4: es la formulación con la que el objetivo del proyecto está planteado en el documento metodológico principal, y no depende de ninguna medición pendiente. ED^2P ($w = 2$) se reporta como **análisis de sensibilidad secundario** en Fase 4 – útil para mostrar si la conclusión cualitativa (el agente ahorra energía sin degradar tiempo de forma inaceptable) es robusta a cuánto se penaliza el retardo, pero nunca como la métrica que decide si el agente "funciona". Fijar esto ahora, antes de tener datos de Fase 4, es precisamente lo que evita la selección posterior de métrica.

11. Tabla maestra: los 24 parámetros, estado real y evidencia

La Tabla 10 recoge, en un único lugar, el estado real de los 24 parámetros originalmente identificados en el reporte anterior (ARC-88), incluyendo los que este documento y sus extensiones (ARC-89/90/91) no tocan directamente – para que ningún parámetro quede implícito o se confunda “mencionado en el resumen” con “auditado en este documento”. La columna **Evidencia** indica dónde vive la justificación real de cada uno; cuando es este documento, se referencia la sección; cuando es el reporte anterior (medición de campaña real, no barrido dedicado) o no existe medición, se indica explícitamente.

#	Parámetro	Valor	Estado	Evidencia
1	interval_ns (CPU)	1 ms	Medido (este doc.)	Sección 3 – valor razonable y conservador, verificado en esta plataforma
2	gpu_interval_ns	5 ms	Medido (este doc.)	Sección 4 – viable y conservador; no aislado como único óptimo frente a 10 ms
3	target_windows_per_rep50ition (CPU)		Medido (este doc.), re-encuadrado	Sección 5 – guardarraíl de duración mínima, nunca activo; no es prueba de suficiencia estadística
4	target_windows_per_rep50ition (GPU)		Medido (este doc.), re-encuadrado	Sección 5 – misma salvedad que #3, más la resolución NVML de ~100 ms (Sección 4)
5	running_ratio_min	0,90	Medido (reporte anterior)	<i>Reporte_Justificacion_Parametros_Experimentales.m</i> §2.3 – valor real observado = 1,0 en el 100 % de las corridas; sin barrido dedicado en este documento

#	Parámetro	Valor	Estado	Evidencia
6	repetitions_per_combination	4	Medido (este doc.), con límite documentado	Sección 5 – mínimo operativo razonable, no garantía de detectar anomalías infrecuentes ni de precisión estadística para energía/EDP en Fase 4
7	D03_TOLERANCE_FRACTION (CPU)	40 %	Medido (este doc.), guardarrail	Sección 9 – confirmado conservador frente al ruido real; no derivado estadísticamente
8	Tolerancia cruzada GPU	5 %	Parcial	Sección 9 – margen confirmado frente al ruido; detección end-to-end pendiente de permiso P4
9	Umbral de estabilidad CAL-10	5 % CV	Sin justificar de origen	<i>Reporte_Justificacion_Parametros_Experimentales.m</i> §2.7 – razonable en la práctica, sin cita propia; no tocado en este documento
10	CV_THRESHOLD_PCT (calentamiento)	5 %	Medido (este doc.)	Sección 7 – robusto en la mayoría de kernels, con excepciones reales documentadas (<i>rodinia_lud</i> , <i>npb_cg</i>); antecedente de Georges et al. recontextualizado (JVM, no HPC nativo)
11	MARGIN (post-detección)	×1,2	Parcial	Sección 7 – barrido cuantifica el efecto en segundos (0,01–0,73 s según kernel), pero sin evidencia de que ×1,2 sea suficiente ni excesivo; decisión de ingeniería pendiente de validación estadística entre repeticiones
12	min_mean_floor (piso GPU)	5,0 % util	Medido (este doc.)	Sección 7 – confirmado del lado correcto con margen en los 3 kernels GPU elegibles; el codo exacto cae en (2,5] %, no resuelto con mayor precisión
13	plateau_ratio (change-point)	0,8	Parcial, método no activado	Sección 7 – ninguno de los 6 kernels de la muestra activó el método de respaldo por punto de cambio; el valor sigue respaldado solo por evidencia histórica de ARC-86, barrido dirigido pendiente
14	min_relative_gain / max_depth (change-point)	0,10 / 6	Sin justificar, NO cubierto por este documento	<i>sensitivity_warmup_detector.py</i> pasa estos valores fijos, nunca los varía – no confundir con los 4 parámetros sí barridos en #10-13
15	_adaptive_window_size	máx(3, mín(5, $n_j/4$))	Sin justificar	Heurística propia sin barrido de sensibilidad; no tocado en este documento
16	Niveles de frecuencia F0–F4	5 puntos equiespaciados + REF	No medible hoy	Sección 10 – bloqueado por permiso P1/P4 pendiente, condición de cierre declarada
17	SAFETY_MARGIN (timeout)	×3,0	Medido (reporte anterior)	<i>Reporte_Justificacion_Parametros_Experimentales.m</i> §2.12 – presupuesto real usado 1–22 %; no tocado en este documento

#	Parámetro	Valor	Estado	Evidencia
18	<code>timeouts_seconds</code> base	ready=15, run=180, shutdown=15	Sin justificar, bajo impacto	<i>Reporte_Justificacion_Parametros_Experimentales.m</i> §2.13
19	Umbral de carga externa E08	1,0	Justificado por convención, no integrado en producción	Sección 10 – valor razonable por convención, pero <code>run_campaign_preflight</code> no lo ejecuta y <code>run_reduced_preflight</code> (que sí lo incluye) nunca es invocado por <code>campaign.py</code> – el chequeo no corre hoy en ninguna campaña real
20	Rango de temperatura E02	[0, 90] °C	Parcial, no integrado en producción	Sección 10 – 90 °C no coincide con el T_{case} de 81 °C documentado para el SKU exacto (Xeon Gold 5315Y); además, <i>Manifest</i> no declara <code>package_temperature_c</code> , así que el chequeo – aunque se invocara – aprobaría siempre por sensor ausente
21	<code>delegated_cpus</code>	6 núcleos	Parcial	<i>Reporte_Justificacion_Parametros_Experimentales.m</i> §2.16 – topología justificada, el conteo exacto no; no tocado en este documento
22	<code>smt_policy</code>	1 hilo/núcleo físico	Medido (este doc.)	Sección 6 – IPC cae 40,4 % con SMT completo, sin cambio en MPKI/LLC (descarta más tráfico de caché; contención de pipeline es la causa compatible, no la única aislada – el diseño cambia <code>OMP_NUM_THREADS</code> a la vez que SMT)
23	Lanzamientos <code>ncu</code> por kernel	Varía, 5–200	Medido (este doc.)	Sección 8 – criterio de convergencia declarado; <code>rodinia_lud</code> cerrado con extensión a 200
24	Peso w del EDP (Fase 4)	$w = 1$ primario, $w = 2$ sensibilidad	Fijado por diseño (este doc.)	Sección 10 – pre-registrado antes de Fase 4 para evitar selección posterior de métrica

Cuadro 10: Tabla maestra de los 24 parámetros identificados en ARC-88, con su estado real y ubicación de la evidencia tras ARC-89/90/91.

De los 24: **10 fueron medidos con barrido dedicado en este documento y sus extensiones** (#1,2,3,4,6,7,10,12,22,23 – incluyendo el cierre nuevo de `smt_policy` y de la convergencia de `rodinia_lud`; esto no significa que ninguno tenga matices – #3/#4 quedan reencuadrados como guardarraíl, no prueba de suficiencia estadística, #6 documenta un límite real de detección de anomalías, #7 es guardarraíl conservador y no tolerancia derivada, y #22 tiene causa parcialmente aislada – ver cada sección para el matiz exacto), **2 provienen de medición directa ya reportada en el documento anterior** sin barrido adicional aquí (#5,17), **5 quedan parcialmente justificados con condición de cierre declarada** (#8 tolerancia cruzada GPU, #11 MARGIN –

cuantificado pero no validado como suficiente, #13 `plateau_ratio` – método de respaldo nunca activado en este barrido, #20 rango de temperatura E02, y #21 `delegated_cpus` – topología justificada, conteo exacto no), **2 quedan justificados por decisión metodológica o convención, pero además con un hallazgo de integración pendiente** (#16 F0-F4 bloqueado por permiso pendiente; #19 E08 por convención de sistemas, pero ni E08 ni E02 ni el preflight de GPU se ejecutan hoy en ninguna campaña real – ver Sección 10), **1 queda fijado por diseño para evitar sesgo de selección posterior** (#24, peso del EDP), y **4 permanecen explícitamente sin justificar** (#9, #14, #15, #18), sin haber sido tocados por este documento ni por su predecesor – listados aquí precisamente para que no desaparezcan de la vista por no formar parte del alcance de ningún barrido hecho hasta ahora.

12. Balance final

Este informe corrió ocho campañas de barrido dedicadas en `paccaA100`, separadas por completo del árbol de resultados de las campañas reales de construcción del catálogo (`hyperion-results/sweeps/` vs. `hyperion-results/campaigns/`), para auditar con evidencia directa los parámetros de diseño experimental de la Fase 1. La Tabla 11 desglosa la contabilidad exacta de corridas, separando **corridas válidas retenidas para el análisis de invocaciones inválidas ya reemplazadas** – estas últimas no se descartan del registro histórico: son precisamente la evidencia de los dos defectos de infraestructura documentados en este informe (`dgemm_n2048/libopenblas.so.0`, y el video truncado de `rodinia_heartwall`).

Cuadro 11: Contabilidad de invocaciones del harness, por barrido.

Barrido	Válidas	Inválidas reemplazadas	Cálculo (válidas)
<code>interval_ns</code> (Sección 3)	105	15 (<code>dgemm_n2048</code>)	7 kernels $\times$ 5 cadencia
<code>gpu_interval_ns</code> (Sección 4)	120	15 (<code>rodinia_heartwall</code>)	8 kernels $\times$ 5 cadencia
<code>repetitions</code> (Sección 5)	150	20 (10 <code>dgemm_n2048</code> + 10 <code>rodinia_heartwall</code>)	15 kernels $\times$ 10 reps
<code>smt_policy</code> (Sección 6)	10	0	2 políticas $\times$ 5 reps
Confirmación aleatorizada (Sección 2)	24	0	18 CPU + 6 GPU
Total	409	50	

En total, el harness se invocó **459 veces** (409 válidas + 50 inválidas reemplazadas) a lo largo de estos ocho barridos. Todas las cifras de este informe (medias, CV%, tablas) usan únicamente las 409 corridas válidas – las 50 inválidas se muestran aquí por transparencia de auditoría, no porque contribuyan al análisis. Aparte de estas, se perfilaron 24 combinaciones independientes con `ncu` (21 del barrido F original + 3 de la extensión de `rodinia_lud`) y se corrieron 50 calibraciones repetidas – ninguna de las dos usa `runner.run_single()`, así que no se suman al total de corridas del harness. Los reintentos por fallos de infraestructura (`dgemm_n2048` por `libopenblas.so.0` ausente, `rodinia_heartwall` por el video truncado) **reemplazan** corridas ya contadas en su barrido original – no se cuentan aparte, para no inflar el total con corridas que en su momento fueron inválidas y después se repitieron. La Sección 11 recoge el estado real de los 24 parámetros identificados originalmente; este balance resume las conclusiones cualitativas, con el cuidado de no reclamar más de lo que cada medición sostiene.

Valor razonable y conservador, verificado en esta plataforma – la formulación correcta para la mayoría de estos parámetros, distinta de “valor óptimo demostrado”: `interval_ns` (1 ms es el primer punto donde el riesgo de jitter severo cae de forma abrupta y el IPC ya es estable, sin

que esto excluya que otros valores cercanos también serían válidos, Sección 3); `gpu_interval_ns` (5 ms es viable y conservador, pero la resolución real del sensor NVML ($\sim 100\text{--}120\text{ ms}$) hace que el margen efectivo sobre el objetivo de ventanas sea mucho más estrecho de lo que sugiere contar lecturas crudas, y 10 ms también sería viable, Sección 4); `repetitions_per_combination` (3 es un mínimo operativo razonable para Fase 1, no una garantía estadística – el reanálisis combinatorio completo sobre las 120 ternas posibles de cada kernel muestra que el resultado depende de qué terna se ejecute, y que la limitación es más seria para la futura evaluación energética de Fase 4, Sección 5); y `target_windows_per_repetition` (un guardarraíl de duración mínima nunca activo en 150 corridas, no una prueba de suficiencia estadística de la muestra resultante).

Confirmado con medición directa, sin matices adicionales: el número de lanzamientos de `ncu` por kernel (con un criterio de convergencia declarado de antemano, y con la extensión dirigida que cierra el caso más exigente, `rodinia_lud`, en 200 lanzamientos, por encima de su punto de convergencia real de 150, Sección 8).

Efecto observado con medición directa, causa y ventaja energética no aisladas: `smt_policy` – en `npb_cg`, la configuración de 12 hilos con SMT completo reduce el IPC agregado en 40,4 % frente a 6 hilos en 6 núcleos físicos, un efecto sistemático y no ruido de medición (Sección 6). Pero el diseño cambió `OMP_NUM_THREADS` (6 $\rightarrow$ 12) a la vez que la política de SMT, así que MPKI y tasa de fallos de LLC constantes descartan más tráfico de caché por unidad de acceso, pero no descartan mayor tráfico total por unidad de tiempo, ni aíslan contención de *pipeline* de otras causas (sincronización de OpenMP, ancho de banda de memoria). Tampoco se midió energía, y el tiempo de pared fue menor bajo SMT completo – así que este resultado no permite afirmar que `one_thread_per_physical_core` sea la política energéticamente superior. Se mantiene por consistencia de telemetría en Fase 1, no como óptimo de EDP confirmado.

Confirmado en dirección, no en magnitud extrema: la confirmación aleatorizada de los dos casos frontera (Sección 2) tiene fuerza distinta en cada dominio. En GPU, `rodinia_backprop` reprodujo exactamente 17 y 9 muestras bajo orden aleatorizado – una confirmación fuerte, cifra por cifra. En CPU, la confirmación solo reprodujo una diferencia moderada en la media de CV % entre 0,5 ms y 1 ms para 3 kernels – no reapareció la cola severa (el máximo de 8,448 % que es el argumento central contra 0,5 ms en la Sección 3), porque esa cola depende de un evento ocasional de interferencia del planificador que 3 repeticiones por punto no garantizan capturar. La confirmación aleatorizada de CPU respalda la *dirección* del hallazgo (0,5 ms es más ruidoso), no una reconfirmación del riesgo de cola extrema en sí.

Guardarraíl conservador confirmado, no tolerancia derivada estadísticamente: `D03_TOLERANCE_FRACTION` (40 %, CPU) – el ruido real medido está casi tres órdenes de magnitud por debajo, pero eso confirma que el guardarraíl nunca se ha acercado a su límite, no que 40 % sea el valor óptimo frente a 10 % o 20 %. La tolerancia cruzada de GPU (5 %) queda **parcialmente justificada**: tiene el mismo margen frente al ruido, pero su capacidad real de detectar una aplicación fallida de frecuencia sigue sin poder verificarse hasta el permiso P4 (Sección 9).

Justificados por decisión metodológica explícita, no por medición (Sección 10): los niveles de frecuencia F0-F4 (sin permiso de escritura disponible todavía) y el umbral de carga externa E08 (razonable por convención de sistemas). El rango de temperatura E02 queda **parcial y corregido**: la hoja de especificaciones exacta del SKU del nodo (Xeon Gold 5315Y) documenta un T_{case} máximo de 81 °C, no 90 °C, así que 90 °C debe presentarse como margen de seguridad conservador frente al rango típico de T_{jmax} de la familia, no como un umbral verificado contra el datasheet exacto de este procesador – una corrección respecto a una afirmación más fuerte en una versión anterior de

este análisis.

Hallazgo de integración, más serio que la falta de barrido de E08/E02: revisando el código directamente (no solo el diseño conceptual), se encontró que **ni E08 ni E02 se ejecutan hoy en ninguna campaña real del proyecto**, y que el preflight de GPU (G01–G03) tampoco corre porque el manifiesto de referencia GPU declara `gpu.enabled: false`. `run_campaign_preflight` (invocado una vez, al inicio de la campaña, desde `cli.py`) no incluye E02 ni E08; ambos viven en `run_reduced_preflight`, pensado para correr antes de cada combinación, pero que `campaign.py` nunca llama (sí verifica E06, procesos ajenos, antes de cada combinación – pero no E02 ni E08). Y aunque se llamara, **Manifest** no declara `package_temperature_c`, así que E02 aprobaría siempre por sensor ausente; y aunque `gpu.enabled` fuera `true`, `cli.py` no le pasa ningún inspector NVML a G01–G03, así que tampoco podrían verificar procesos CUDA ajenos, modo persistente ni configuración MIG. Esto no es una limitación del alcance de este informe – es un hallazgo sobre el instrumento de producción mismo, documentado en detalle en la Sección 1 y en la Sección 10, con la misma seriedad que el defecto de datos de `rodinia_heartwall` descrito más abajo. Por alcance de un proyecto de pregrado, este informe documenta el hallazgo y lo deja como prerrequisito explícito antes de la campaña DVFS definitiva – integrar estos tres chequeos en `campaign.py` es trabajo de implementación fuera del alcance de esta auditoría de parámetros, no algo que este informe deba resolver.

Fijado por diseño para evitar sesgo de selección posterior: el peso w del EDP se pre-registra en $w = 1$ (EDP clásico) como métrica primaria de Fase 4, con $w = 2$ (ED^2P) como análisis de sensibilidad secundario – decidir esto *antes* de tener resultados de Fase 4 es lo que evita, aunque no fuera la intención original, escoger implícitamente la métrica que favorezca al agente de control (Sección 10).

Limitación metodológica declarada, no oculta: los barridos principales corrieron sus combinaciones en orden fijo, no aleatorizado, sin preflight repetido de carga externa/temperatura por combinación – un riesgo real en un nodo compartido. Una confirmación dirigida y aleatorizada de los dos casos frontera más citados reprodujo el mismo resultado (Sección 2), pero eso acota el riesgo solo para esos dos casos, no para la totalidad de los barridos.

Hallazgo no buscado, con impacto real fuera del alcance original de este informe: se encontró y corrigió un defecto de datos de producción – `rodinia_heartwall` corría, desde algún momento posterior a su caracterización original (ARC-86), contra un video sintético truncado a 25 cuadros en vez de los 20000 esperados, sin que el harness lo detectara (el binario de Rodinia termina con código de salida 0 tras su propio error de validación de argumentos). Esto invalidaba silenciosamente tres repeticiones ya marcadas **accepted** en la campaña de producción y los resultados de `rodinia_heartwall` en dos de los barridos de este mismo informe. Se corrigió la causa raíz, se recuperaron los datos de producción, y se re-corrieron los barridos afectados. Este episodio es, en sí mismo, la mejor evidencia del valor de este ejercicio de auditoría: no se estaba buscando este problema, y no se habría encontrado sin medir de verdad en vez de asumir que los datos ya existentes eran correctos. La misma disciplina de verificación llevó, en una revisión posterior de este informe, a corregir una segunda afirmación demasiado fuerte sobre `rodinia_heartwall` y sobre el rango de temperatura E02, y a descubrir que la resolución real del sensor NVML es mucho más gruesa de lo que el conteo ingenuo de lecturas sugería.

Alcance y límites del informe, dichos sin rodeos: este trabajo valida que la instrumentación y varios parámetros de adquisición funcionan de forma reproducible en `paccaA100`, y justifica decisiones locales de ingeniería para construir el dataset de Fase 1. No demuestra que estos parámetros sean

universales para otras plataformas. No valida todavía el efecto de DVFS, el ahorro energético ni el EDP final, porque faltan las campañas reales multifrecuencia (bloqueadas por los permisos P1/P4). Tampoco convierte automáticamente las ventanas recolectadas en un dataset estadísticamente suficiente para entrenar un modelo – eso deberá verificarse en Fase 2 con partición por corrida/carga, balance de clases y evaluación fuera de muestra, un análisis que este informe no realiza.

De los 24 parámetros originalmente identificados, la clasificación canónica es la de la Tabla 10 (Sección 11) – una versión anterior de este balance daba un conteo distinto (12/3/4) que no coincidía con esa tabla; se corrige aquí para que ambos sean la misma fuente. Ninguno de los 24 queda sin veredicto explícito: **10** quedan medidos con barrido dedicado en este informe y sus extensiones sin matices adicionales, **2** provienen de medición directa ya reportada en el documento anterior, **5** quedan parcialmente justificados con una condición de cierre concreta (tolerancia cruzada GPU, `MARGIN`, `plateau_ratio`, rango de temperatura E02, y `delegated_cpus`), **2** quedan justificados por decisión metodológica o convención explícita (F0-F4, E08), **1** queda fijado por diseño para evitar sesgo de selección posterior (peso del EDP), y **4** permanecen explícitamente sin justificar, sin haber sido tocados por ningún barrido hasta ahora. La diferencia frente a la primera versión de este informe no es cuántos parámetros tienen veredicto – ya eran todos – sino qué tan fuerte es la palabra que describe cada veredicto: “razonable y verificado en esta plataforma” en vez de “óptimo” o “confirmado” donde la evidencia no alcanza para lo segundo.

Referencias

- [1] Ghazanfar Ali, Mert Side, Sridutt Bhalachandra, Nicholas J. Wright, and Yong Chen. An automated and portable method for selecting an optimal gpu frequency. *Future Generation Computer Systems*, 149:71–88, 2023. doi: 10.1016/j.future.2023.07.011.
- [2] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous java performance evaluation. In *Proc. 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA '07)*, pages 57–76, 2007. doi: 10.1145/1297027.1297033.
- [3] João Guerreiro, Aleksandar Ilic, Nuno Roma, and Pedro Tomás. Dvfs-aware application classification to improve gpgpus energy efficiency. *Parallel Computing*, 83:93–117, 2019. doi: 10.1016/j.parco.2018.02.001.
- [4] James H. Laros III, Kevin Pedretti, Suzanne M. Kelly, Wei Shu, Kurt Ferreira, John Van Dyke, and Courtenay Vaughan. Energy delay product. In *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science, pages 51–55. Springer, London, 2013. doi: 10.1007/978-1-4471-4492-2_9.